

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

TIÊU LUN CÔNG NGHỆ PHẦN CỨNG TRONG DỮ LIỆU VÀ GIẢI THUẬT

**HỆ THỐNG QUẢN LÝ HỆ SINH VIÊN VÀ ENGINE TRONG SƠ LƯỢC NGÀNH THUẬT
TOÀN (DSA) – RECORDS-AND-DECISION ENGINE**

Giảng viên hướng dẫn: ThS. Bô Văn Vinh (T-Bao)

Mã lớp học phần: DASA230179_03

Nhóm thực hiện: Nhóm 12

Sinh viên thực hiện:

Họ và tên	MSSV
Bùi Tấn Phôt (Nhóm trưởng)	25110289
Nguyễn Phan Thịnh Tín	25110361
Nguyễn Thành	25110335
Nguyễn Minh Khang	25110227

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN CÔNG NGHỆ PHẦN MỀM



HCMUTE

TIÊU LUN CÔNG NGHỆ PHẦN CỤ TRÚC DỮ LIỆU VÀ GIẢI THUẬT

**HỆ THỐNG QUẢN LÝ HỆ SINH VIÊN VÀ ENGINE TRONG SƠ LƯỢC NGÀNH THUẬT
TOÀN (DSA) – RECORDS-AND-DECISION ENGINE**

Giang viên hướng dẫn: ThS. Bô Văn Vinh (T-Bao)

Mã lớp học phần: DASA230179_03

Nhóm thực hiện: Nhóm 12

Sinh viên thực hiện:

Họ và tên	MSSV
Bùi Tấn Phôt (Nhóm trưởng)	25110289
Nguyễn Phan Thịnh Tín	25110361
Nguyễn Thành	25110335
Nguyễn Minh Khang	25110227

L I C M N

Li u tiển, Nhúm 12 chũng em xin chón thịnh gi li cm n sớu sc nht n Ban Gióm hiu Trng i hc Cũng ngh K thut TP. H Chờ Minh cụng toịn th quỳ Thy Cũ ò to iu kin hc tp thun li vị trang b cho chũng em nhng nn tng tri thc quỳ bõu.

c bit, chũng em xin gi li tri ón chón thịnh n Thy ThS. Bõ V ớnh (T-Bao) — ging viổn ph trỏch hc phn Cu trũc D liu vị Gii thut (Mổ lp hc phn: DASA230179_03). Trong sut 14 tun hc tp va qua, Thy ò luũn mang n cho chũng em nhng bị ging tóu huyt, sinh ng vị giũu tồnh thc tin. S ng hnh, tn tớnh hng dn, lng nghe vị gii ỏp mi vng mc t Thy ò giũp chũng em thu hiu sớu sc bn cht mũn hc cng nh nh hng rũ rựng trong sut quỏ trỡnh xóy dng òn.

Chũng em xin trón trng tip thu mi li nhn xổt, úng gúp quỳ giỏ t Thy khũng ngng hoịn thĩn òn, trau di t duy gii thut vị phỏt trin vng chc còc k nng chuyổn mũn cho chng ng phỏa trc. Kờnh chũc Thy luũn di dỏo sc khe, trịn y nim vui vị luũn gt hỏi c nhiu thịnh cũng trong s nghip ging dy!

Mt ln na, Nhúm 12 chũng em xin chón thịnh cm n Thy!

TP. H Chờ Minh, ngỳ 28 thỏng 09 nm 2026

Sinh viổn thc hĩn

Nhúm 12

BNG ÒNH GIÒ VỊ PHÓN CŨNG NHIM V

Mã lp hc phn: DASA230179_03

Nhúm thc hin: Nhúm 12

Ging viốn hng dn: ThS. Bo V ớnh (T-Bao)

STT	H vị tổn / MSSV	Nhim v phón cũng	Mc hoịn thịnh
1	Búi Tn Phòt (Nhúm trng) MSSV: 25110289	Iu phi chung, tng hp bào cồ; Thit k vị t cị t Bng bm úng (Closed Hash Table), hịm bm FNV-1a phc v yốu cu bt buc MC1 (Tra cu chồnh xồ theo MSSV).	100%
2	Nguyñ Phan Thịn Tin MSSV: 25110361	Thit k vị t cị t Cóy ng cc i (Custom Binary Max-Heap), gii thut Floyd Build-Heap $O(N)$ phc v yốu cu bt buc MC2 (Truy xut sinh viốn cú GPA cao nh).)	100%
3	Ngũ c Thịn MSSV: 25110335	Thit k vị cị t cu trức ch mc lp ClassIndexFilter phc v yốu cu t phòt hin RQ1 (Truy xut vị lc danh sồch sinh viốn theo Lp); tham gia o c benchmark.	100%
4	ng oịn Minh Khang MSSV: 25110227	Thit k vị cị t mng con tr sp xp GPA (SortedGpaFilter), thut toòn Tỏm kim Nh phón 2 u biốn phc v yốu cu t phòt hin RQ2 (Truy vn theo khong GPA); tồch hp h thng.	100%

ònh giò kt qu: Cồc thịnh viốn hoịn thịnh cồc nhim v ững thi hn, hoịn thịnh nhim v tt. Mc dư cú chửt khú khn trong quồ trớnh tỏm hieu cồc kin thc nhng c s h tr t cồc bn trong nhúm, cúng lp ỗ hoịn thịnh òn ững thi hn.

TUYỂN B QUYN S HU THỊNH PHN - D3

Tuấn th quy nh v ònh giò nng lc cò nhón theo Mc 10, 11 v 19 ca bị, tng thịnh viồn xóc nhn quyn s hu k thut, trồch nhim cị t vị lp lun thit k c lp nh sau:

Thịnh viồn & MSSV	Thịnh phn s hu	úng gúp k thut chònh
Búi Tn Phòt (Nhúm trng) MSSV: 25110289	Module 1: Closed Hash (Phc v MC1)	T cị t Bng bm úng (Closed Hash Table) v hịm bm FNV-1a v Linear Probing, tra cu chònh xóc theo MSSV trong thi gian trung bnh $O(1)$.
Nguyñ Phan Thịnh Tin MSSV: 25110361	Module 2: Custom Heap (Phc v MC2)	T cị t Cọy ng cc i (Binary Max-Heap) v thut toòn Floyd Build-Heap $O(N)$, trồch xut sinh viồn cú GPA cao nht trong thi gian $O(1)$.
Ngũ c Thịnh MSSV: 25110335	Module 3: Class Filter (Phc v RQ1)	Thit k vị cị t ch mc lp ClassIndexFilter, lc danh sòch sinh viồn theo Lp nhanh chúng v thi gian truy vn trung bnh $O(1)$.
ng oịn Minh Khang MSSV: 25110227	Module 4: Sorted GPA (Phc v RQ2)	Xóy dng mng con tr sp xp GPA v gii thut Tỏm kim Nh phón hai u biổn, truy vn theo khong im $[minGPA, maxGPA]$ trong thi gian $O(\log N)$.

Cam kt cò nhón: Toịn b còc thịnh viồn cam kt hieu sỏu sc mỗ ngun thịnh phn do mớnh s hu, nm vng lý thuyt phc tp vị sn sịng gii thờch, trace code c lp trong búi vn òp (D8).

PHN M U	1
1. L do chn ti v bi cnh the tin ca bi ton	1
2. Mc tiu nghin cu v chun u ra hc phn (CLO Alignment)	1
3. i tng, phm vi nghin cu v tp d liu th nghim	2
4. Phng php tip cn, cng c the hin v b cc bo co	2
CHNG 1: TNG QUAN BI TON V THIT K CU TRC D LIU	4
1.1. M t bi ton v phn tch nghip v h thng	4
1.1.1. Bi cnh bi ton Records-and-Decision Engine	4
1.1.2. M hnh ha the th d liu v cc trng thuc tnh	4
1.1.3. Phn tch cc m the truy cp d liu v thao tc ch o	5
1.2. Xc nh v m hnh ha cc yu cu bt buc chung	5
1.2.1. Yu cu MC1: Tra cu chnh xc theo nh danh quy m ln	5
1.2.2. Yu cu MC2: Truy xut phn t theo th t v mc u tin cc tr	6
1.3. Phn tch v pht hin cc yu cu c th ca lnh vc	6
1.3.1. Yu cu t pht hin 1: Lc d liu theo iu kin ch mc	6
1.3.2. Yu cu t pht hin 2: Truy vn d liu theo khong gi tr	7
1.4. Phn tch xung t yu cu v gii php x l k thut	7
1.4.1. Xc nh cp yu cu xung t ct li trong h thng	7
1.4.2. So snh hng Kt hp cu trc v nh i c ch ch	7
1.4.3. Bin minh phng n k thut gii quy t xung t ca nhm	8
1.5. p dng khung nh gi 7 tiu ch v quy trnh la chn Q1–Q4	8
1.5.1. Ma trn so snh 7 tiu ch k thut trn cc cu trc ng vin	8
1.5.2. p dng quy trnh Q1–Q4 cho yu cu tra cu chnh xc MC1	10
1.5.3. p dng quy trnh Q1–Q4 cho yu cu trch xut cc tr MC2	11
1.5.4. p dng quy trnh Q1–Q4 cho cc yu cu t pht hin	11
1.6. Bin minh chi tit vic la chn cu trc d liu ct li	11
1.6.1. Bin minh la chn Bng bm ng (Closed Hash Table) cho MC1	11

1.6.2. Bin minh la chn Cy ng cc i (Custom Max-Heap) cho MC2	12
1.6.3. Bin minh cu trc d liu b tr cho cc truy vn theo khong	13
1.7. Phn tch v gii trnh cc phng n k thut b loi b	13
1.7.1. L do loi b gii php duy t tyn tn timer Mng tun t	13
1.7.2. L do loi b Bng bm m (Chaining) v Danh sch lin kt	13
1.7.3. L do loi b Cy tm kim nh phn thng thng (Unbalanced BST)	13
1.8. Thit k kin trc tng th 3 tng ca h thng	14
1.8.1. Tng giao din ngi dng (Presentation Layer)	14
1.8.2. Tng cu trc d liu ct li v thut ton (DSA Core Layer)	14
1.8.3. Tng lu tr bn vng (Persistence Layer) v tn phn tch	14
1.8.4. Cu trc t chc th mc m ngun d n	14
CHNG 2: HIN THC H THNG, THC NGHIM V I SNH HIU NNG . .	17
2.1. Ci t chi tit cu trc Bng bm ng t u (From Scratch 1)	17
2.1.1. Thit k nh v cu trc Entry	17
2.1.2. Hin thc hm bm chui FNV-1a v phn b gi tr bm	18
2.1.3. C ch gii quy t ng v thut ton tm kim	19
2.1.4. Phn tch chi ph thi gian v khng gian b nh	22
2.2. Ci t chi tit cu trc Cy ng cc i t u (From Scratch 2)	22
2.2.1. Bui din mng phng 1 chiu v cng thc truy xut ch s nt	22
2.2.2. Hin thc thut ton vun ng Floyd Build Heap trong thi gian $O(N)$	23
2.2.3. Ci t cc thao tc truy xut cc tr v cp nht ng	25
2.2.4. X l quy tc phn nh ng hng (Tie-breaking Rule)	26
2.3. Ci t cc thut ton b tr v x l Range Query	27
2.3.1. Xy dng mng d liu sp xp theo im GPA (RQ2)	27
2.3.2. Hin thc thut ton tm kim nh phn 2 u bin trn mng sp xp . . .	27
2.3.3. Hin thc b lc theo m lp hc phn (RQ1) bng Class Index View	29
2.4. Hin thc tng lu tr bn vng v c ch np d liu hng lot	30

2.4.1. Thit k cu trc lu tr file d liu (JSON / CSV)	30
2.4.2. C ch np d liu hng lot (Bulk Load) vo b nh RAM	30
2.4.3. m bo nguyn tc c lp gia lu tr v x l truy vn	32
2.5. Xy dng giao din tng tc v trc quan ha d liu	32
2.5.1. Thit k giao din dng lnh CLI tng tc nhanh	32
2.5.2. Xy dng module trc quan ha cu trc d liu v benchmark	32
2.6. Thit k kch bn kim th t ng v x l trng hp bin	32
2.6.1. Xy dng b Unit Test t ng cho cc cu trc t ci t	32
2.6.2. Kim th x l d liu trng lp, d liu rng v trn ngng	33
2.6.3. Kim th tnh n nh khi chy ti truy vn lin tc	33
2.7. Thec ngim o c Benchmark hiu nng trn tp d liu quy m ln	33
2.7.1. Mi trng phn cng, h iu hnh v phng php lun o lng	33
2.7.2. Kt qu o thi gian tra cu MC1 qua cc mc d liu ln	36
2.7.3. Kt qu o thi gian trch xut cc tr MC2 v xy dng ng	39
2.7.4. Kt qu o thi gian thec thi cc truy vn t pht hin	39
2.8. i snh hiu nng thec ngim vi l thuyt v thut ton tuy ntnh	42
2.8.1. So snh tc Bng bm ng ($O(1)$) vi Qut tuy ntnh ($O(N)$)	42
2.8.2. So snh tc truy xut Max-Heap ($O(1)$) vi Qut tm Max tuy ntnh ($O(N)$)	43
2.8.3. nh gi chi tit mc tiu hao b nh RAM v hiu ng Cache	43
2.9. Nht k g li (Debug Log) v ti u ha gii thut	44
2.9.1. Ghi nhn li pht sinh khi x l iu kin dng tm kim trn Bng bm . .	44
2.9.2. Ti u ha cp pht b nh v bo ton con tr trong Bulk Load	44
2.10. Tuy n b Trch nhim Gii trnh v S dng AI v Lim chnh Hc thut (AI-Audit Log)	45
PHN KT LUN V BI HC KINH NGHIM	47
1. Kt lun tng quan v d n	47
2. Bi phn t c nhn ca 4 thnh vin (Individual Reflections - D7)	47

2.1. Phn t ca sinh vin Bi Tn Pht (MSSV: 25110289)	47
2.2. Phn t ca sinh vin Nguyn Phan Thnh Tin (MSSV: 25110361)	48
2.3. Phn t ca sinh vin Ng c Thnh (MSSV: 25110335)	48
2.4. Phn t ca sinh vin ng on Minh Khang (MSSV: 25110227) . .	49
TI LIU THAM KHO	50

DANH SÁCH HÌNH V

Hnh 1:	Cu trc thc th Student	18
Hnh 2:	Cu trc nh Entry trong Bng bm	18
Hnh 3:	M ngun hm bm chui FNV-1a 64-bit t ci t	19
Hnh 4:	M ngun thut ton np d liu v x l ng tuy n tn h trong Bng bm . .	20
Hnh 5:	M ngun gii thut tm kim tra cu chnh xc theo MSSV trn Bng bm	21
Hnh 6:	S tun t (Sequence Diagram) lung tra cu chnh xc theo MSSV trn Bng bm	21
Hnh 7:	M ngun thut ton Floyd Build Heap $O(N)$ t ci t	23
Hnh 8:	M ngun hm heapifyDown duy tr bt bin cu trc Max-Heap . .	25
Hnh 9:	S tun t (Sequence Diagram) lung trch xut GPA cao nht trn Max-Heap	26
Hnh 10:	Hin thc ton t phn nh ng hng kt hp GPA v MSSV	26
Hnh 11:	M ngun xy dng mng d liu sp xp theo GPA phc v Range Query	27
Hnh 12:	M ngun thut ton Binary Search 2 u bin trch xut khong GPA	28
Hnh 13:	S tun t (Sequence Diagram) lung lc sinh vin theo khong GPA	29
Hnh 14:	S tun t (Sequence Diagram) lung khi to v np d liu hng lot (Bulk Load)	31
Hnh 15:	Benchmark MC1 tra cu MSSV ti u di d liu	37
Hnh 16:	Benchmark MC1 tra cu MSSV ti gia di d liu	37
Hnh 17:	Benchmark MC1 tra cu MSSV ti cui di d liu	38
Hnh 18:	Benchmark MC1 tra cu MSSV khi kha khng tn ti trong h thng	38

Hnh 19:	Biu Benchmark MC2 trch xut GPA cao nht trn Max-Heap so vi Baseline	39
Hnh 20:	Benchmark RQ1 lc danh sch sinh vin lp C01 (u di)	40
Hnh 21:	Benchmark RQ1 lc danh sch sinh vin lp C13 (Gia di)	40
Hnh 22:	Benchmark RQ1 lc danh sch sinh vin lp C25 (Cui di)	41
Hnh 23:	Benchmark RQ1 khi tra cu m lp khng tn ti	41
Hnh 24:	Dashboard minh ha truy vn RQ2 vi khong GPA [8.0, 9.5]; s liu C++ Core trong Bng 3 s dng workload [8.0, 10.0]	42

DANH MỤC TỪ VIẾT TẮT

Ký hiệu	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
DSA	Data Structures and Algorithms	Cấu trúc Dữ liệu và Giải thuật
MSSV	Student Identifier	Mã số sinh viên (Khóa chèn nh danh)
GPA	Grade Point Average	Điểm trung bình tích lũy học tập
MC	Mandatory Criteria	Yêu cầu nghiệp vụ bắt buộc tối thiểu
MC1	Mandatory Criteria 1	Tra cứu chèn xóa sinh viên theo MSSV
MC2	Mandatory Criteria 2	Truy xuất sinh viên có GPA cao nhất
RQ	Required Query	Yêu cầu truy vấn mở rộng tìm kiếm
RQ1	Requirement 1	Truy xuất danh sách sinh viên theo Lớp
RQ2	Requirement 2	Truy xuất sinh viên theo không GPA
FNV-1a	Fowler–Noll–Vo (Version 1a)	Giải thuật băm phi mã hóa tốc độ cao
RAM	Random Access Memory	Bộ nhớ truy xuất ngẫu nhiên không biến
CPU	Central Processing Unit	Bộ vi xử lý trung tâm
JSON	JavaScript Object Notation	Định dạng dữ liệu hoàn toàn văn bản thuần
STL	Standard Template Library	Thư viện mẫu chuẩn trong ngôn ngữ C++
CLO	Course Learning Outcomes	Chuẩn đầu ra tích hợp các học phần
CRUD	Create, Read, Update, Delete	Bộ thao tác dữ liệu cơ bản

PHN M U

1. Lý do chn tị vị bi cnh thc tin ca bii toàn

Trong cũng tộc qun lý vị khai thóc thũng tin hc tp ti còc c s giòo dc i hc quy mũ ln, vic lu tr, tra cu vị x lý nhanh chúng khi lng h s sinh viỏn ngiy cing tr nỏn cp thit. H thng thũng tin qun lý khũng ch dng li chc nng lu tr d liu tnh, mị cùn cn hot ng nh mt h thng x lý h s vị h tr ra quy t nh (*Records-and-Decision Engine*) cú kh nng phn hi tc thờ còc tộc v vn hnh ct lủi: tra cu thũng tin sinh viỏn theo mỗ nh danh duy nht, xp hng vị truy xut sinh viỏn cú kt qu hc tp cao nht xỏt duy t hc bng, cng nh phón loi vị thng kỏ danh sỏch sinh viỏn theo còc tiỏu chờ im s hoc lp hc.

Khi quy mũ d liu m rng lỏn mc thit k ln gm 10.000.000 sinh viỏn, còc phng phỏp duy t tun t tuyen tỏnh ($O(N)$) bc l hn ch rủ rt v tr, gỏy chim dng tị nguyỏn CPU vị lịm gim tc phn hi. Vic nghiỏn cu vị ng dng còc cu trũc d liu chuyỏn sỏu trong b nh chỏnh (In-memory Data Structures) nh Bng bm úng (Closed Hash Table) vị Cỏy ng cc i (Max-Heap) lị gii phỏp quan trng nhm tị u thi gian x lý mt s tộc v trng yu xug mc $O(1)$ hoc $O(\log N)$ túy theo loi thao tộc vị cu trũc d liu c s dng. Xut phỏt t yỏu cu thc tin ú, Nhúm 12 la chn tị: **”H thng Qun lý H s Sinh viỏn vị Engine i sỏnh Hiu nng Thut toàn (DSA)”** nhm xỏy dng gii phỏp k thut hoịn chnh, ỏp ng cón bng gia hiu nng thi gian vị khũng gian b nh trỏn tp d liu thc nghi m hin tị 10.000.000 bn ghi.

2. Mc tiỏu nghiỏn cu vị chun u ra hc phn (CLO Alignment)

òn c xỏy dng bóm sỏt theo còc chun u ra tỏch hp ca hc phn Cu trũc D liu vị Gii thut (Mỗ lp hc phn: DASA230179_03):

Th nht, i vị chun u ra CLO1 (Mũ hnh húa bii toàn vị lp lun tỏnh ửng n), nhúm tin hnh phón tỏch bi cnh bii toàn thc t, xỏc nh chỏnh xỏc tp d liu u vọ (Input), u ra (Output), rịng buc d liu (Constraints) vị bao ph còc trng hp biỏn c bit (Edge Cases).

Th hai, i vị chun u ra CLO3 (Cị t vị kim th cu trũc d liu ct lủi), nhúm t tay

cải thiện hiệu suất (*from scratch*) trở nên dễ dàng với C++ hai cú trúc dữ liệu tổng quát:
 Bảng băm sử dụng hàm băm FNV-1a vì kỹ thuật dò tìm tuyến tính (Linear Probing) và
 Cây nhị phân (Custom Binary Max-Heap) và giải thuật tìm kiếm Floyd $O(N)$, không sử dụng
 các thuật toán cấu trúc dữ liệu sẵn có của STL C++.

Th ba, i vi chun u ra CLO7 (Phón tồch cu trũc phón cp vị ònh giò suy gim hiu nng), nhúm cị t cu trũc cóy phón cp (Binary Heap), phón tồch c ch duy trơ bt bin ng vị ònh giò còc iu kin suy gim hiu nng trong quò trnh vn hình thc t.

Th t, i vi chun u ra CLO8 (Capstone – La chn, ònh giò vị bo v cu trũc d liu), nhúm phón tồch sớ còc mũ thc truy cp d liu (Access Patterns), òp dng khung 7 tiớu chớ so sòngh vị quy trnh 4 cớu hi quy t nh Q1–Q4 gii trnh phng òn la chn cu trũc, lim rũ còc ònh i k thut (*trade-offs*) vị còc gii phòp b loi b.

3. i tng, phm vi nghiõn cu vi tp d liu th nghim

Vị trí ghi nhớ của, tập trung vào các cấu trúc dữ liệu có bản vị nóng cao (Mảng, Bảng băm, Cây nhị phân, Mảng xếp) cũng cần ghi nhớ tìm kiếm, xếp vị trí của truy vấn trong bộ nhớ RAM.

V phạm vi nghiên cứu, hệ thống tập trung xử lý các thao tác gm: tra cứu chờn xóc theo MSSV (MC1), tìm sinh viên có GPA cao nhất (MC2), lọc sinh viên theo Mã Lp (RQ1) và lọc danh sách sinh viên theo không GPA (RQ2).

V tp d liu th nghim, ln o C++ Core hin ti np data/database.json lĩm d liu gc, sau ú bulk-add trong RAM t 10.000.000 bn ghi sinh viĩn. óy lĩ quy mũ ln quan sòt rũ rt s khĩc bit gia cĩc ng i thut toĩn.

4. Phng phòp tip cn, cũng c thc hin vi b cc bào cò

V phng phòp tip cn, nghiỏn cu kt hp gia lý thuyt phón tồch phc tp thi gian/khũng gian (O, Ω, Θ) vi phng phòp thc nghiim o c Benchmark bng ng h o thi gian phón gii cao ca th vin chun C++ (`std::chrono::high_resolution_clock`).

V cũng c vị mĩi trng phòt trin, tng x lý thut toà n ct lủi c xóy dng bng
ngũn ng C++ (chun C++20). Tng giao din vị trc quan hóa d liu c xóy dng bng
React/Next.js kt hp Ant Design, giao tip vi C++ Engine thũng qua giao thc API

cu ni (DSA Bridge).

V b cc bào cào, tị liu c t chc thnh 2 chng trng tóm cng danh mc tị liu tham kho: Chng 1 trnh bñ tng quan bñ toøn, mĩ hnh húa d liu, phón tồch xung t yỏu cu v ị òp dng quy trnh Q1–Q4 bin minh cu trũc. Chng 2 trnh bñ chỉ tit hnh thc mĩ ngun C++ ca hai cu trũc ct lủi t u, kin trũc phón tng, kim th trng hp biỏn v ị toịn b s liu Benchmark thc nghim i sòn trổn tp d liu hnh ti 10.000.000 sinh viỏn, kổm nht kỳ g li v ị danh mc tị liu tham kho chuyỏn ngnh.

CHNG 1: TNG QUAN BỊ TOÀN

VỊ THIT K CU TRỮ D LIU

1.1. Mũ t bị toàn vị phón tồch nghip v h thng

1.1.1. Bi cnh bị toàn Records-and-Decision Engine

Trong cũng tồc qun lý ịo to i hc, còc h thng thũng tin phi tip nhn vị x lý khi lng ln d liu phòt sinh theo tng hc k. H thng qun lý h s sinh viỏn cn òp ng vai trò ca mt h thng x lý h s vị h tr ra quyt nh (*Records-and-Decision Engine*) trong thi gian thc. H thng nịy trc tip gii quyt còc bị toàn nghip v thng gp: tra cu nhanh h s cò nhón ca sinh viỏn theo mỗ s, trồch xut danh sòch sinh viỏn cú thnh tồch hc tp cao nht xỏt khen thng, cng nh phón loi hc lc sinh viỏn theo còc di im phc v cũng tồc theo dủi kt qu hc tp.

Khi dung lng d liu sinh viỏn t mc 10.000.000 bn ghi, còc phng phòp duy t tun t tuyn tồnh ($O(N)$) bc l rủ s suy gim hiu nng, góy lõng phờ chu k CPU vị kỏo dị thi gian phn hi. Vớ vy, vic thit k vị cị t còc cu trữ d liu phứ hp trong b nh chồnh (In-memory Data Structures) úng vai trò quan trng nhm ti u thi gian x lý mt s truy vn trng yu, cú th t mc $O(1)$ hoc $O(\log N)$ tỳy theo loi thao tồc vị cu trữ d liu c s dng.

1.1.2. Mũ hnh hóa thc th d liu vị còc trng thuc tồnh

Mi h s sinh viỏn trong h thng c chun hóa cht ch thũng qua cu trữc thc th Student gm 4 trng thuc tồnh c bn nhm mũ phng sòt thc t qun lý ịo to.

Th nht, trng id (Mỗ s sinh viỏn – MSSV) lị chui kỳ t s gm 8 ch s, úng vai trò lị khóa chồnh duy nht (*Unique Primary Key*) phón bit gia còc sinh viỏn, nm trong di giò tr t 25150001 n 35150000 i vi tp benchmark 10.000.000 bn ghi.

Th hai, trng name lu tr h vị tỏn y ca sinh viỏn di dng chui kỳ t chun.

Th ba, trng classId lu tr mỗ nh danh lp hc phn trc thuc. Trong tp d liu gc 500.000 bn ghi, mỗ lp c phón b trỏn 25 lp t C01 n C25; khi bulk-add lỏn 10.000.000 bn ghi stress test, b sinh d liu to thỏm còc mỗ lp dng 23DTH01, 25KTP02, ... nỏn min classId ca tp benchmark 10M rng hn tp gc.

Thứ t, trng gpa lu tr im trung bnh tồch ly ca sinh vión di dng s thc du phý ng chõnh xõc kõp (double) trong thang im t 0.0 n 10.0 (phón b thc nghiõm trong di t 4.0 n 10.0). Cu trũc thc th nịy c úng gúi ti u gĩm thiũ dung lng chim dng b nh m khi nps lng ln bn ghi vĩa RAM.

1.1.3. Phón tồch cõc mũ thc truy cý d liũ vĩa thao tồc ch o

Da trõn phón tồch hĩnh vi vñ hĩnh thc t, mũ thc truy cý d liũ (Access Patterns) ca h thng c chia thĩnh hai nhũm thao tồc rũ rt: thao tồc ghi (*Write Path*) vĩa thao tồc c (*Read Path*).

Thao tồc ghi trong phm vi benchmark din ra tý trung gĩa on nps d liũ ban u (*Bulk Loading*) vĩa khi lng thc nghiõm hĩn ti 10.000.000 bn ghi trong b nh chõnh. Cõc phõp o MC1, MC2, RQ1 vĩa RQ2 c thc hĩn trõn nh chýp d liũ õ nps xong; phn CRUD ca Web Dashboard lĩ chc nng minh ha qũn lý d liũ vĩa c tồch khi tps liũ hĩu nng. Mc 10.000.000 bn ghi c dũng trc tĩp cho nh Dashboard vĩa bng s benchmark cý nhĩ.

Ngc li, h thng c thĩt k theo mũ hõnh read-heavy, trong ú cõc thao tồc c chim t trng ch yu trong quõ trõnh vñ hĩnh, din ra liõn tc vĩa tn sut cao vĩa ùi hĩ phn hĩ tc thõ. Cõc thao tồc c ch o bao gm: tra cu chõnh xõc thũng tin mĩt sinh vión theo MSSV (MC1), truy xũt ngay lp tc sinh vión cú im GPA cao nhĩt (MC2), lc sinh vión theo mĩ lp (RQ1) vĩa trõch xũt danh sõch sinh vión theo khõng im GPA (RQ2). Do ú, mc tiõu thĩt k lĩ ti u hĩa ng i truy vñ ca cõc thao tồc c.

1.2. Xõc nh vĩa mũ hõnh hĩa cõc yõu cu bt buc chung

1.2.1. Yõu cu MC1: Tra cu chõnh xõc theo nh danh quy mũ ln

Yõu cu bt buc th nhĩt (MC1 – *Exact Match Lookup*) ùi hĩ h thng phi tĩp nhĩn mĩt khĩa tĩm kim lĩ MSSV vĩa tr v chõnh xõc, toĩn vñ toĩn b h s sinh vión tng ng, hoc thũng bõo khũng tĩm thỹ nu khĩa khũng tn ti trong c s d liũ.

c t k thũt ca yõu cu MC1 xõc nh: u vĩa lĩ chũi id cĩn tra cu; u ra lĩ con tr hoc tham chĩu n i tng Student tha mĩn; Rĩng buc hĩu nng bt buc lĩ thĩ gian thc thĩ phi t mc tĩm cĩn trung bnh $O(1)$ c lp vĩa kõch thc d liũ $N = 10.000.000$.

Còn trong hợp biến (Edge Cases) cần xử lý y bao gồm việc tra cứu khóa năm u đi d liu (25150001), gia đi d liu (không 30150000), cuối đi d liu (35150000) vị còn trong hợp tra cứu khóa hoàn toàn không tồn tại trong hệ thống.

1.2.2. Yêu cầu MC2: Truy xuất phần tử theo thứ tự vị mức ưu tiên các trường

Yêu cầu bắt buộc thứ hai (MC2 – *Extreme Priority Retrieval*) đòi hỏi hệ thống phải trích xuất ngay lập tức thông tin cá nhân của sinh viên có điểm trung bình học tập (gpa) cao nhất hoàn toàn trong nhóm phục vụ hiệu quả thi đua khen thưởng vị xã hội công bằng.

Các đặc tính của yêu cầu MC2 xóc nh: ưu vị trí yêu cầu trích xuất các id hoàn toàn; ưu vị trí điểm sinh viên có gpa lớn nhất; Quy tắc giải quyết ngang hàng (*Tie-breaking Rule*) của nhóm quy định nghiêm ngặt: nếu có nhiều sinh viên cùng điểm trung bình GPA cao nhất (về điểm trung bình 10.0), hệ thống sẽ ưu tiên chọn sinh viên có MSSV nhỏ hơn theo thứ tự tăng dần; Riêng việc hiệu năng lập trình thao tác lấy phần tử các trường phải có thời gian $O(1)$ mỗi khi không có phép duyệt list hoàn toàn 10.000.000 phần tử hay thực hiện thao tác sắp xếp list hoàn toàn bằng thuật toán sắp xếp vì chỉ phép $O(N \log N)$.

1.3. Phân tích vị phân tích các yêu cầu của thứ tự các lĩnh vực

1.3.1. Yêu cầu phân tích 1: Lấy dữ liệu theo id khóa học

Tổng hợp nghiệp vụ quản lý học tập thực tế, ban chấp hành khoa vị các vấn đề tập trung cần truy xuất danh sách hoàn toàn sinh viên thực hiện một lớp học cụ thể (Yêu cầu RQ1 – *Filter by Class ID*).

Các đặc tính của yêu cầu này là trong khóa học classId cụ thể phân nhóm các sinh viên theo nhóm học tập. Trong tập benchmark gốc, 500.000 sinh viên có phân bố trở lại 25 lớp; khi mở rộng/bulk-add lên 10.000.000 bản ghi, cần lập sinh ra một số lượng lớn các nhóm học tập. Trong code hiện tại, RQ1 build trực tiếp các classId -> indexes; với việc truy vấn theo lớp không cần quét list hoàn toàn danh sách vị không sao chép các id của Student. Chỉ phép build list $O(N)$, còn query trung bình list $O(1)$ lấy view vị chỉ phân sinh $O(K)$ khi tăng giá trị cần liệt kê học serialize kết quả.

1.3.2. Yêu cầu tối ưu hình 2: Truy vấn dữ liệu theo không giới hạn

Hệ thống xử lý bảng, phân tích vị trí các nhân viên kim loại theo xu hướng các trường xuất danh sách sinh viên theo các địa điểm học tập bình quân (Yêu cầu RQ2 – *Range Query by GPA*), với địa điểm Xuất sắc [9.0, 10.0], Giỏi [8.0, 8.9], học Khỏi [6.5, 7.9].

Ý tưởng để thực hiện truy vấn theo không gian giới hạn là sử dụng cây phân đoạn. Cấu trúc dữ liệu phục vụ yêu cầu này bắt buộc phải duy trì các thông tin cần thiết, cho phép xác định vị trí biên dưới (minGPA) và vị trí biên trên (maxGPA) trong thời gian logarit $O(\log N)$. Nếu cần tạo range view cho truy vấn dạng $O(\log N)$; nếu cần duyệt/hình thức tổng cộng K sinh viên thỏa mãn thì tổng chi phí sẽ là $O(\log N + K)$.

1.4. Phân tích xung đột yêu cầu vị trí phân tích kỹ thuật

1.4.1. Xác định các yêu cầu xung đột tiềm ẩn trong hệ thống

Trong quá trình thiết kế hệ thống, nhóm nhận diện một xung đột kỹ thuật nghiêm trọng giữa hai nhóm yêu cầu: Yêu cầu truy vấn phân tích theo khóa ngẫu nhiên (MC1) và Yêu cầu truy vấn theo thứ tự tiến học theo không gian giới hạn (MC2 và RQ2).

Bản chất của Bảng băm (Hash Table) là sử dụng hàm băm phân tán ngẫu nhiên các khóa vào các ô nhớ nhằm giảm thiểu nguy cơ va chạm truy cập $O(1)$ trung bình. Tuy nhiên, phân tích các yêu cầu phân tích hoặc tổng hợp theo thứ tự tiến học của dữ liệu, khiến Bảng băm phân tán tổng hợp không thể hình thức thao tác tìm kiếm giới hạn nhất hay truy vấn theo không gian không duyệt qua toàn bộ M ô nhớ ($O(N)$). Ngược lại, các cấu trúc duy trì thứ tự như Cây phân tích tìm kiếm cân bằng (AVL, Red-Black) hoặc Mảng sắp xếp (Sorted Array) cho phép giải quyết tất cả các yêu cầu truy vấn không gian ($O(\log N + K)$), nhưng lại không thể bỏ qua truy vấn MSSV mỗi $O(1)$ và chi phí phân bổ/xóa lớn $O(N)$ khi thay đổi.

1.4.2. So sánh hai Kiểu cấu trúc và đánh giá các lựa chọn

Nhóm đã tiến hành phân tích và so sánh hai kiểu tiếp cận giải quyết xung đột theo chu kỳ kết quả:

Loại tiếp cận 1 (Đánh giá các lựa chọn – *Deliberate Trade-off*): Chỉ là lựa chọn duy nhất một cấu trúc dữ liệu (chẳng hạn mảng sắp xếp tổng hợp) phục vụ mọi yêu cầu. Hướng tiếp cận này giúp tối

kim b nh do ch duy trờ mt bn sao d liu, nhng buc h thng phi chp nhn ònh i hieu nng tra cu khi tởm kim trỏn thuc tởnh khũng phi lị khúa sp xp chỏnh, gỏy suy gim hieu nng khi s lng truy vn MC1 tng cao.

Hng tip cn 2 (Kt hp cu trũc – *Composition Approach*): Thit k h thng a cu trũc ng b, trong ú mi cu trũc d liu chuyỏn bit m nhn ỹng bũ toỏn s trng ca mớnh. Trong cũi t hin ti, HashTable lu con tr const Student* tr v vũng d liu gc tr li bn ghi nhanh; SortedGpaFilter cng lu mng con tr ỏ sp xp theo GPA vị tr v range view, nh ú trỏnh sao chỏp Student khi lc khong; riỏng CustomMaxHeapGpaFinder duy trờ bn sao vector<Student> ni b n gin húa quỏ trỏnh vun ng.

1.4.3. Bin minh phng ònh k thut gii quy t ca nhúm

Nhúm 12 quy t nh la chn gii phỏp **Kt hp cu trũc chuyỏn bit (Composition Approach)**. C th, h thng tỏch hp bn cu trũc/ch mc b tr trong b nh: Bng bm úng (Closed Hash Table) phc v MC1 t $O(1)$ trung bớnh; Cỏy ng cc i (Custom Binary Max-Heap) phc v MC2 t $O(1)$ khi c nũt gc sau khi ỏ build; Class Index View phc v RQ1 bng ònh x classId -> vector<int> vị query trung bớnh $O(1)$; vị Mng con tr sp xp theo GPA (Sorted Pointer Array) phc v RQ2 t $O(\log N)$ cho pha tởm biỏn vị $O(1)$ to view kt qu. Thit k nũy chp nhn ònh i b nh: HashTable vị SortedGPA tit kim bng cỏch lu con tr v d liu gc, ClassIndex lu ch s sinh viỏn theo lp, trong khi Heap sao chỏp Student cũi t thao tỏc vun ng rũ rựng hn.

1.5. Ờp dng khung ònh giò 7 tiỏu chỏr vị quy trỏnh la chn Q1–Q4

1.5.1. Ma trn so sỏnh 7 tiỏu chỏr k thut trỏn cỏc cu trũc ng viỏn

Nhúm tin hỡnh so sỏnh bn cu trũc d liu ng viỏn theo 7 tiỏu chỏr k thut liỏn quan trc tip n cỏc yỏu cu ca h thng: Mng thũ tun t (*Raw Dynamic Array*), Bng bm úng (*Closed Hash Table*), Cỏy ng cc i (*Binary Max-Heap*) vị Mng d liu ỏ sp xp (*Sorted Array*).

Ỗ ngha vị vai trũ ca tng tiỏu chỏr k thut trong h thng c xỏc nh c th nh sau:

Tiỏu chỏr th nht (Truy xut theo khúa nh danh – *Key Lookup*): ònh giò kh nng tởm kim chỏnh xỏc mt h s sinh viỏn khi bit trc mỗ s sinh viỏn (MSSV). óy

lệ thao tác trung yếu của MC1, thì hiệu suất tìm kiếm là $O(1)$ trung bình.

Tiểu chờ thứ hai (Truy xuất theo vị trí – *Positional Access*): ở đây ta truy cập một phần tử khi biết trước chính xác vị trí của nó trong mảng. Các cấu trúc dữ liệu chuyển đổi mảng thành danh sách liên kết có thể đạt $O(1)$ nhờ cách tính toán địa chỉ offset trực tiếp chuyển thành ghi phần tử.

Tiểu chờ thứ ba (Chèn xóa vị trí – *Insertion and Deletion*): ở đây chỉ cần tìm vị trí cần chèn/xóa trong danh sách liên kết để thực hiện thao tác. Các cấu trúc như bảng băm có thể đạt $O(1)$ hoặc $O(\log N)$, trong khi mảng đơn giản thì cần $O(N)$.

Tiểu chờ thứ tư (Duyệt tổng hợp – *Total Order*): ở đây ta cần duyệt tất cả các phần tử theo một thứ tự nhất định (như GPA hay MSSV) để tìm kiếm. Không cần phải lưu trữ toàn bộ dữ liệu vì chỉ cần $O(N \log N)$.

Tiểu chờ thứ năm (Truy vấn theo khoảng – *Range Query*): ở đây ta cần trả lời nhanh chóng tập hợp K phần tử thỏa mãn điều kiện nào đó trong một đoạn $[min, max]$. Đây là tiểu chờ quyết định độ phức tạp của các truy vấn RQ2.

Tiểu chờ thứ sáu (Trò chơi ưu tiên – *Extreme Priority Retrieval*): ở đây ta cần trả lời khi danh sách ưu tiên có phần tử cần xóa hoặc thêm. Có thể dùng cấu trúc heap để đạt $O(1)$.

Tiểu chờ thứ bảy (Ổn định vị trí tìm kiếm – *Asymptotic Stability*): ở đây ta cần đo lường độ sai lệch giữa các phần tử trong tập trung bình vị trí của chúng khi thêm/xóa phần tử mới.

Mà trên sơ đồ phân loại 7 tiểu chờ kể trên có 4 cấu trúc dữ liệu cơ bản đang được đề cập:

Tiểu chò	Mng thũ	Closed Hash	Max-Heap	Mng Sort (GPA)	Cu trúc ti u
1. Khóa (Key)	$O(N)$	$O(1)$ tb	$O(N)$	$O(N)$	Bng bm (MC1)
2. V trờ	$O(1)$	—	$O(1)$	$O(1)$	Mng thũ / Sort
3. Chỗn / Xúa	$O(1) / O(N)$	Build/chỗn $O(1)$ tb; delete khũng h tr trc tip trong index tnh	$O(\log N)$	$O(N)$	Heap / Hash
4. Th t	$O(N \log N)$	—	$O(N \log N)$	$O(N)$	Mng sp xp
5. Khong	$O(N)$	—	—	$O(\log N)$ view / $O(\log N + K)$ lit kổ	Mng con tr sp xp (RQ2)
6. Cc tr	$O(N)$	$O(N)$	$O(1)$ nh	$O(1)$ biổn	Max-Heap (MC2)
7. n nh	$O(N)$	$O(1)/O(N)$	$O(\log N)$	$O(\log N)$	Phi hp a cu trúc

Bng 1. Ma trn so sònh 7 tiỏu chỏ k thut trổn 4 cu trúc d liu ng viổn

1.5.2. Ồp dng quy trỏnh Q1–Q4 cho yỏu cu tra cu chỏnh xỏc MC1

Quy trỏnh 4 cúu hi quy t nh (Q1–Q4 Decision Framework) cho yỏu cu MC1 c ồp dng cú h thng nh sau:

Th nh t, i vi cúu hi Q1 (Thao tỏc cú cn duy trỏ th t d liu hay khũng?): Yỏu cu tra cu chỏnh xỏc theo MSSV hoỉn toỉn khũng cn duy trỏ bt k th t nộ gia còc phn t, ch cn tỏm kim khp khóa chỏnh xỏc.

Th hai, i vi cúu hi Q2 (Khóa tra cu thuc loi gỏ v i mũ hỏnh khi lng cũng vic ra sao?): Khóa l i chui kỳ t s 8 ch s phỏn b u. Khi lng cũng vic thuc dng c chim u th sau giai on np ban u.

Th ba, i vi cúu hi Q3 (H thng cú chp nhn ri ro trng hp xu nh t $O(N)$ hay khũng?): Ri ro ng ỏm v i suy gim hieu nng c gim ờng k nh hỉm ỏm FNV-1a v i

duy trở h s tỉ thp ($\alpha \approx 0.50$).

Th t, i vì cóu hi Q4 (Còc yu t phi tìm cn vị phn cng nh hng th nịo?): Bng bm úng dng mng phng liỏn tc cú li cho tồnh nh v khũng gian ca b m CPU Cache Line vị cú kh nng gim Cache Miss so vì danh sòch liỏn kt.

1.5.3. Òp dng quy trỏnh Q1–Q4 cho yỏu cu trỏch xut cc tr MC2

Quy trỏnh 4 cóu hi quy t nh cho yỏu cu MC2 c òp dng nh sau:

Th nht, i vì cóu hi Q1: Nghip v trỏch xut sinh viỏn cú GPA cao nht ch cn duy trở th t cc b tỉ nũt gc (phn t cc i), hoịn toịn khũng cn sp xp toịn phn c tp d liu 10.000.000 bn ghi.

Th hai, i vì cóu hi Q2: Tn sut truy vn cc tr đin ra liỏn tc, trong khi vic xóy dng ng ch cn thc hìn ửng mt ln lũc khi ng h thng.

Th ba, i vì cóu hi Q3: H thng yỏu cu tc phn hi t $O(1)$ cho thao tồc truy xut phn t cc i tỉ nũt gc, chỉ phờ cp nht li ng khi cú bin ng lị $O(\log N)$ m bo tồnh n nh cao.

Th t, i vì cóu hi Q4: Cu trũc Cóy ng nh phón c biu đin trc tip trỏn mng 1 chiu, khũng cn lu còc con tr cóy (*Left, Right*) cho tng node, cho phỏp truy cp ch s cha-con bng còc phỏp tồnh ch s cú chỉ phờ $O(1)$.

1.5.4. Òp dng quy trỏnh Q1–Q4 cho còc yỏu cu t phòt hìn

i vì yỏu cu RQ2 (Lc sinh viỏn theo khong GPA): Cóu hi Q1 xòc nh d liu bt buc phi c sp xp theo th t im GPA tng dn. Cóu hi Q2 xòc nh mìn giò tr tòm kim lị liỏn tc. Cóu hi Q3 vị Q4 ch ra rng cu trũc Mng con tr ò sp xp kt hp thut toòn Tòm kim nh phón hai u biỏn (Lower Bound vị Upper Bound t cị t) mang li hìu nng $O(\log N)$ cho pha xòc nh khong, i li chỉ phờ lu thỏm N con tr Student*.

1.6. Bin minh chỉ tit vic la chn cu trũc d liu ct lủi

1.6.1. Bin minh la chn Bng bm úng (Closed Hash Table) cho MC1

Nhúm quy t nh t cị t cu trũc Bng bm úng (Closed Hash Table vị c ch Open Addressing vị Linear Probing) cho yỏu cu MC1 da trỏn ba lý do k thut nn tng:

Thứ nhất, Bảng băm ứng lưu trữ các dữ liệu trực tiếp trong một mảng phẳng liên tục, trình bày cụ thể như sau: các nút liên kết, giúp tìm kiếm bảng băm trở thành dữ liệu 10.000.000 bản ghi.

Thứ hai, như cấu trúc mảng liên tục, Bảng băm ứng tận dụng tối đa bộ nhớ cache. Khi xảy ra xung đột, bước duyệt tuần hoàn (step = +1) thông qua truy cập các dữ liệu liên kết, có lợi hơn so với việc nhảy con trỏ ngẫu nhiên trong bảng băm.

Thứ ba, việc tổ hợp hàm băm FNV-1a (Fowler-Noll-Vo 1a) 64-bit giúp phân bố đều các giá trị băm trong không gian băm. Code chọn lọc thực nghiệm theo công thức $M = 2N + 1$, duy trì hệ số tỉ lệ $\alpha \approx 0.50$. Ví dụ benchmark hình thức $N = 10.000.000$, bảng có $M = 20.000.001$ ô. Dung lượng này cần chọn chu kỳ băm thấp; vì linear probing bước +1, tuần hoàn chỉ duyệt các vị trí băm không phụ thuộc vào vị trí M là số lẻ hay số nguyên tố.

1.6.2. Bấm minh họa chọn Có ý nghĩa (Custom Max-Heap) cho MC2

Để vì yêu cầu MC2, Có ý nghĩa của cải tiến (Custom Binary Max-Heap) là chọn phù hợp với:

Thứ nhất, tuần hoàn của Max-Heap mà bỏ qua phần tử cuối tiên liên tục giảm từ vị trí gốc (chỉ số 0), cho phép tránh xuất sinh vòng lặp GPA cao nhất trong thí nghiệm thực tế là $O(1)$.

Thứ hai, thuật toán vun đống Floyd Build Heap cho phép chuyển vị trí các mảng dữ liệu ban đầu thành một Max-Heap hợp lệ trong thí nghiệm duyệt tuần hoàn $O(N)$ vì cũng thực chứng minh:

$$\sum_{h=0}^{\lfloor \log_2 N \rfloor} \frac{N}{2^{h+1}} \cdot O(h) = O(N) \quad (1)$$

Thuật toán này nhanh hơn đáng kể so với việc chuyển đổi liên tiếp phần tử một $O(N \log N)$ hoặc sắp xếp toàn bộ mảng.

Thứ ba, quy tắc phân nhánh theo MSSV cải tiến trực tiếp vào toàn bộ số thành phần ưu tiên (higherPriority), giúp kiểm soát tỉ lệ nút gốc như khi nhiều sinh viên có cùng GPA.

1.6.3. Bin minh cu trức d liu b tr cho còc truy vn theo không

gii quy t yóu cu lc theo không im GPA (RQ2), SortedGpaFilter to mt vector<const Student*> vị sp xp còc con tr nịy theo GPA. Sau khi xóy dng, h thng òp dng Tòm kim nh phón nh v hai v trở biể trong thì gian $O(\log N)$. Kt qu truy vn c biu đin bng GpaRangeViewResult, gm con tr ti mng ò sp xp vị cp ch s $[startIndex, endIndex)$; vớ vy pha tr kt qu khũng sao chố K bn ghi Student.

1.7. Phón tồch vị gii trớnh còc phng òn k thut b loi b

1.7.1. Lý do loi b gii phòp duy t tồnh trởnn Mng tun t

Gii phòp duy t tũyn tồnh trởnn mng thũ ($O(N)$) khũng c la chn lịm phng òn chồnh trởnn tp d liu quy mũ ln ca h thng vớ vị tp d liu 10.000.000 bn ghi, mị thao tồc tra cu n l ùi hi phi thc hin trung bớnh hn 5.000.000 phỏp so sòngh khúa vị trong trng hp xu nht lị 10.000.000 phỏp so sòngh, góy ra tr hịng chc mili-giỏy cho mị truy vn, lịm suy gim kh nng òp ng ca h thng.

1.7.2. Lý do loi b Bng bm m (Chaining) vị Danh sòch liỏn kt

Phng òn Bng bm m (Separate Chaining) s dng danh sòch liỏn kt n ti mị bucket b loi b vớ hai nhc im ct t:

Th nht, vic cp phòt ng hịng triu nũt nh l (Node*) dn n hin tng phón mnh b nh RAM vị tiỏu tn thỏm mt lng ln b nh ph phờ cho còc con tr next (òt nht 8 bytes cho mị nũt).

Th hai, vic duy t theo còc con tr liỏn kt nm phón tòn còc a ch ngu nhiỏn trong vớng nh Heap cú tồnh nh v khũng gian kỏm hn vị cú th lịm tng s ln Cache Miss so vị cu trức mng liỏn tc.

1.7.3. Lý do loi b Cóy tòm kim nh phón thũng thng (Unbalanced BST)

Cóy tòm kim nh phón thũng thng khũng cú c ch t cón bng b loi b do c thứ d liu thc t: khi tp d liu u vớ cú xu hng tng dn theo mớ sinh viỏn, cóy s b suy bin thịnh mt danh sòch liỏn kt vị chiu cao cóy t $O(N)$. Khi ú, mị u th tòm kim logarit b phờ v vị chi phờ thao tồc thòoi húa v mc tũyn tồnh $O(N)$.

1.8. Thit k kin trũc tng th 3 tng ca h thng

1.8.1. Tng giao din ngi dũng (Presentation Layer)

Tng Presentation Layer c xóy dng bng cũng ngh Web hin i (Next.js/React kt hp Ant Design v Tailwind CSS) nhm cung cp giao din trc quan, thón thĩn cho ngi dũng. Tng nĩy cú nhĩm v tip nhĩn tham s tĩm kim t ngi dũng, gi yĩu cu n tng x lý thũng qua API Bridge, vĩ trc quan hĩa kt qu tra cu cng nh cĩc biu i sĩnh hĩu nng Benchmark chĩ tit. Tng nĩy c thĩt k phĩn tĩch rũ rĩng vĩ khũĩng can thĩp vĩo logic tĩnh toĩn giĩ thĩt.

1.8.2. Tng cu trũc d liũ ct lũĩ vĩ thĩt toĩn (DSA Core Layer)

Tng DSA Core Layer ũng vai trũ lĩ tng x lý trung tĩm ca toĩn b h thng, c lp trĩnh bng C++. ỹ lĩ nĩ chĩ cĩc cu trũc d liũ cĩĩ t t u (Bng bm ũng FNV-1a, Cĩy ng cc i Max-Heap, Mng d liũ sp xp theo GPA) vĩ trc tip thĩ logic nghĩp v MC1, MC2, RQ1, RQ2 trong b nh RAM. Cĩc phĩp o c thĩ gian trong phĩn bĩo cĩo c ghi nhĩn tĩ tng nĩy bng `std::chrono::high_resolution_clock`; cĩc nh Dashboard c dũng nh mĩnh ha giao din, khũĩng thay th cho bng s liũ thĩc nghĩm.

1.8.3. Tng lu tr bn vng (Persistence Layer) vĩ tĩnh phĩn tĩch

Tng Persistence Layer m nhĩn vai trũ c vĩ lu tr d liũ thũ ra cĩc tp tin lu tr bn vng (CSV/JSON) trĩĩ cng khi khi ng vĩ kt thũĩc ng dng. Nhũm tuĩn th nguyĩĩn tc phĩn tĩch trĩch nhĩm trong thĩt k h thng: Tng lu tr ch chũ trĩch nhĩm np d liũ thũ vĩo RAM mĩt ln duy nht (*Bulk Load*). Mĩ thao tĩc tĩm kim, lc d liũ vĩ trĩch xĩt cc tr u do tng DSA Core giĩ quyĩt trc tip trĩĩn cĩc cu trũc d liũ t cĩĩ t trong b nh, khũĩng da vĩo cĩc cĩu lĩnh truy vĩn cú sn ca h qũn tr c s d liũ.

1.8.4. Cu trũc t chĩc th mĩc mĩĩ ngũn d ĩĩn

Toĩn b h thng mĩĩ ngũn c module hĩa phĩĩn tng rũ rĩng, chũn hĩa ranh giĩ gia giao din nh nghĩa (interface/) vĩ phĩn mĩĩ hĩn thĩc giĩ thĩt (src/):

code_dsa/

|-- interface/

Tap hop cac header khai bao (.h)

```

| |-- core_crud/          # Module CRUD quan ly sinh vien
| | \-- StudentCRUD.h
| |-- core_hash/          # Module Closed Hash Table (MC1)
| | |-- HashTable.h
| | |-- LinearSearch.h
| | |-- FindStudentById.h
| | \-- Benchmark.h
| |-- core_heap/          # Module Max-Heap (MC2)
| | |-- CustomMaxHeapGpaFinder.h
| | |-- LinearMaxScanGpaFinder.h
| | |-- FindStudentByMaxGpa.h
| | \-- BenchmarkMaxGpa.h
| |-- core_sorted_gpa/    # Module Binary Search theo GPA (RQ1)
| | |-- SortedGpaFilter.h
| | |-- LinearGpaFilter.h
| | |-- FindStudentByGpaRange.h
| | \-- Benchmark.h
| |-- core_class_filter/  # Module Loc theo Lop sinh hoat (RQ2)
| | |-- OptimizedLinearFilter.h
| | |-- LinearFilter.h
| | |-- FindStudentByClassId.h
| | \-- Benchmark.h
| \-- Student.h          # Dinh nghia struct Student
|-- src/                  # Tap hop cac file hien thuc ma nguon (.cpp)
| |-- core_crud/
| |-- core_hash/
| |-- core_heap/
| |-- core_sorted_gpa/
| |-- core_class_filter/

```

```
| \-- Student.cpp
|-- bao_cao/          # Bao cao hoc thuat LaTeX va hinh anh thuc nghiem
|-- web/              # Giao dien Web Dashboard (Next.js/React/Tailwind)
|-- main.cpp          # CLI Menu dieu khien chuong trinh chinh
|-- dsa_bridge.cpp     # IPC WebSocket/HTTP Bridge giao tiep Web-C++
\-- bench_core.cpp    # Cong cu do luong Benchmark tu dong
```

CHNG 2: HIN THC H THNG, THC NGHIM VỊ I SÒNH HIU NNG

2.1. Cũ t chi tit cu trũc Bng bm úng t u (From Scratch 1)

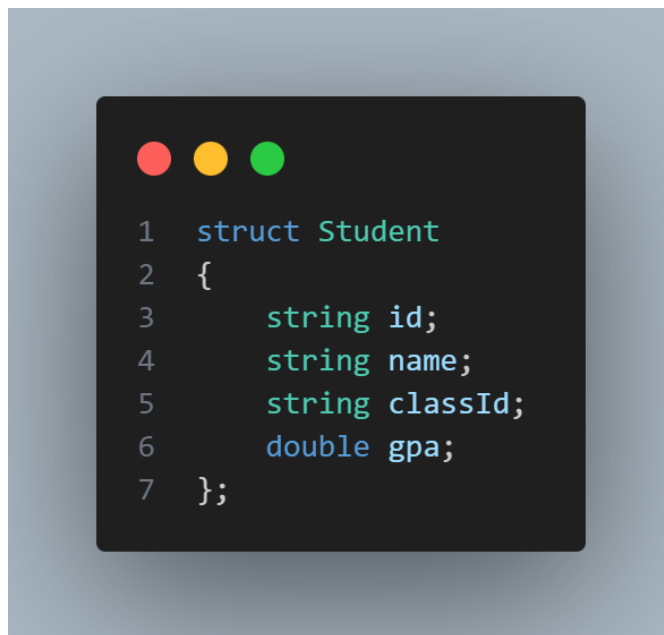
2.1.1. Thĩt k ã nh vị cu trũc Entry

hĩn thc cu trũc Bng bm úng (Closed Hash Table vì c ch Open Addressing vì Linear Probing) t xóy dng t u bng C++20, nhũm thĩt k cu trũc ã nh Entry tĩnh gĩn cho mũ hĩnh bng tra cu tĩnh sau Bulk Load. Mĩ v trũ trong bng bm c nh ngha bĩ cu trũc:

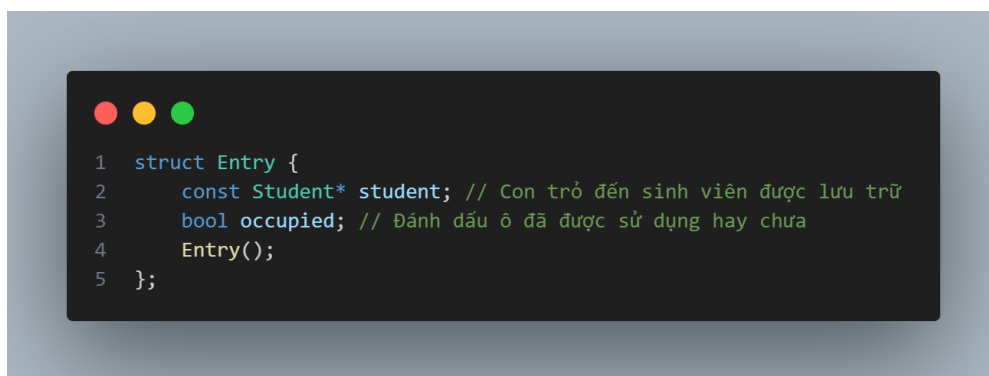
- `const Student* student`: Con tr tr trc tip n bn ghi sinh viĩn gc trong vũng nh chung.
- `bool occupied`: C boolean ònh du ã nh õ c s dng (true) hay cùn trng (false).

V mĩt t chc b nh, i tng Student bao gm 4 trng thũng tĩn: chui mĩ nh danh id, h tĩn name, mĩ lp classId vì im gpa kiu double (8 bytes). Nhũm np d liu trc tip t tp JSON vĩo mĩt mng liĩn tc, sau ú mĩ ã nh Entry trong bng bm ch lu mĩt con tr 8 bytes kĩm 1 byte c trng thĩi (cũng b m cn l 64-bit alignment lĩ 16 bytes), gĩũp tit kim òng k dung lng b nh khi m rng lĩn khĩng 20 triu ã nh.

Trong kch bn vn hĩnh ca òn, d liu sinh viĩn c np hoĩn tt thũng qua tĩn trĩnh Bulk Load vì sau ú h thng ch yu phc v cĩc truy vn tra cu ch mc vì tĩn sut cao (Read-Heavy / Analytical Workload). Do ú, thĩt k c nh phĩn occupied m bo tĩnh n gĩn, hĩu qu vị tĩ u hĩa khũng gian b nh.



Hình 1. Cấu trúc thể th Student



Hình 2. Cấu trúc ã nh Entry trong Bng bm

2.1.2. Hình thể hịbm chuỗi FNV-1a vị phón b giò tr b m

Hịbm ứng vai trò phón tồn u còc khóa MSSV trở tợn b khũng gian bng. Nhúm la chn hịbm FNV-1a (Fowler–Noll–Vo 1a) phiên bn 64-bit [6] vớ u im cị t n gin, tc tồnh tòn nhanh trở chuỗi kỳ t vị cho phón tồn tt trở tợn d liu s bõo danh sinh viõn.

Thut tòn hot ng ãa trở hai hng s tòn hc nguyên t theo c t IETF [6]: hng s bứ ban u $\text{FNV_OFFSET_BASIS} = 14695981039346656037\text{ULL}$ vị hng s nguyên t b m $\text{FNV_PRIME} = 1099511628211\text{ULL}$. Quò trởnh b m duy t tun t tng byte kỳ t ca chuỗi MSSV, thực hìn phõp XOR bit vị giò tr tồch ly hìn ti (value $\wedge = c$) ri nhón vị hng s nguyên t (value $\ast = \text{FNV_PRIME}$).



Hình 3. Mã nguồn hàm băm chuỗi FNV-1a 64-bit tối ưu

Giờ ta băm 64-bit ra thành x vị trí trong bảng thông qua phép chia lấy dư modulo vì kích thước bảng $M = 2N + 1$. Chọn chọn M này nhằm duy trì hệ số tải xấp xỉ 0.5, tức giảm số lần probe trung bình khi duyệt tìm. Trong lần đầu tiên, kích thước bảng thực tế là 20.000.001.

2.1.3. Cách giải quyết xung đột toàn cục

Khi xảy ra xung đột giữa hai khóa cố định chúng ta ban đầu $h_0 = \text{hash}(\text{key}) \pmod{M}$, nhóm phép giải quyết Duyệt tìm (Linear Probing): $h_i = (h_0 + i) \pmod{M}$ với $i = 1, 2, \dots$. Phương pháp này lưu trữ toàn bộ các dữ liệu trong một mảng liên tiếp, tận dụng tính cục bộ không gian (Spatial Locality) của bộ nhớ CPU Cache Line 64-byte [8].

duy trì hiệu suất trung bình tìm kiếm $O(1)$, hệ số tải (*Load Factor*) cần kiểm soát mức $\alpha = N/M \approx 0.500$. Ví dụ benchmark hệ số tải, $\alpha = 10.000.000/20.000.001 \approx 0.500$.

```

1 // Xây dựng Hash Table
2 // Dùng Open Addressing + Linear Probing
3 void HashTable::build(const std::vector<Student>& students){
4     // Capacity  $\approx 2n + 1$  để giảm collision
5     const size_t capacity =
6         students.empty() ? 1 : students.size() * 2 + 1;
7     table.assign(capacity, Entry());
8     collisions = 0;
9     probes = 0;
10    for (const Student& student : students){
11        size_t index = hash(student.id);
12        while (table[index].occupied){
13            ++collisions;
14            index = (index + 1) % table.size();
15        }
16        table[index].student = &student;
17        table[index].occupied = true;
18    }
19 }

```

Hình 4. Mã nguồn thuật toán nạp dữ liệu vị trí lý tưởng tuyến tính trong Bảng băm

Trong thuật toán xây dựng bảng băm (build), khi chèn một sinh viên, thuật toán tìm kiếm vị trí h_0 vị trí tìm kiếm đầu tiên qua bảng băm nếu không tìm thấy vị trí trống (table[index].occupied). Phần tiếp theo sẽ gồm trực tiếp vị trí tiếp theo vị trí bị chiếm (table[index].occupied = true).

Trong việc thao tác tìm kiếm (search), giải thuật bắt đầu từ vị trí h_0 vị trí duyệt qua bảng băm liên tiếp. Nếu gặp vị trí trống (table[index].occupied), thuật toán kết thúc ngay lập tức không trả về vị trí nullptr. Nếu gặp vị trí trống có mã sinh viên trùng khớp (table[index].student->id == targetId), hàm trả về con trỏ sinh viên tương ứng. Nếu không tìm thấy vị trí trống, hàm trả về vị trí cuối cùng của bảng băm M .

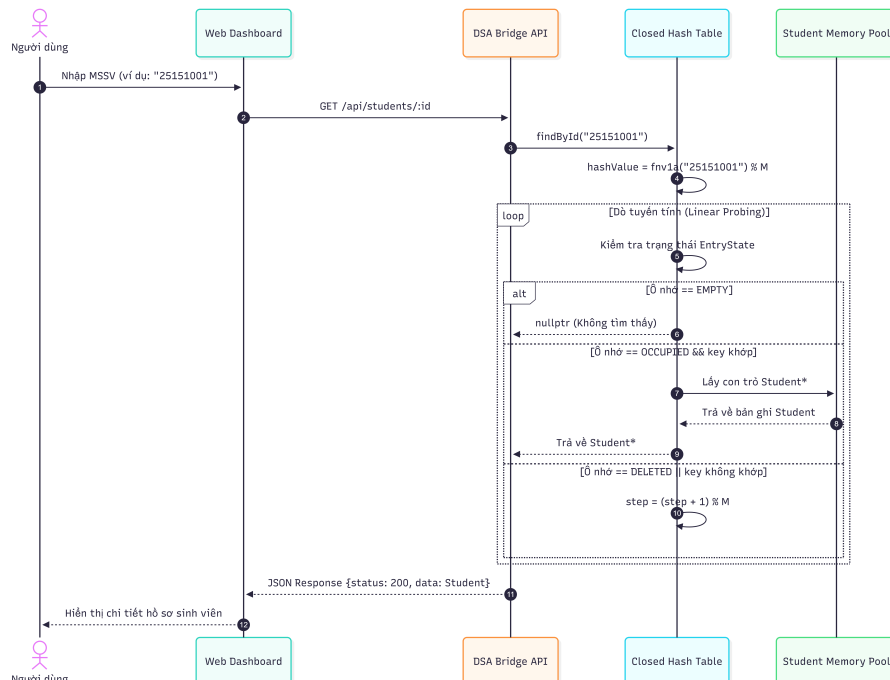
```

1  const Student* HashTable::search(const std::string& targetId){
2      probes = 0;
3      if (table.empty())
4          return nullptr;
5      size_t index = hash(targetId);
6      for (size_t step = 0; step < table.size(); ++step){
7          ++probes;
8          if (!table[index].occupied)
9              return nullptr;
10         if (table[index].student->id == targetId)
11             return table[index].student;
12         index = (index + 1) % table.size();
13     }
14     return nullptr;
15 }

```

Hình 5. Mã nguồn giải thuật tìm kiếm chuỗi theo MSSV trong Bảng băm

Lưu ý hình thức tra cứu chuỗi theo MSSV (Yếu tố MC1) của mô hình hóa trong Sơ đồ Hình 6. Toàn bộ trình diễn: tiếp nhận MSSV từ giao diện Web, chuyển qua API Bridge, thực hiện tra cứu trong Bảng băm ứng dụng bảng FNV-1a với Linear Probing trong thời gian trung bình $O(1)$, sau đó chuyển kết quả thành JSON trả về giao diện.



Hình 6. Sơ đồ (Sequence Diagram) luồng tra cứu chuỗi theo MSSV trong Bảng băm

2.1.4. Phón tồch chi phờ thi gian vị khũng gian b nh

Theo phón tồch toành hc ca Donald E. Knuth [2], vị Bng bm úng s dng dù tuyñ tồnh, s ln kim tra ã nh k vng cho mt thao tồc tòm kim thnh cũng $S(\alpha)$ vị khũng thnh cũng $U(\alpha)$ xp x bng:

$$S(\alpha) \approx \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right) = \frac{1}{2} \left(1 + \frac{1}{1-0.5} \right) = 1.50 \ln \text{ truy cp} \quad (2)$$

i vị thao tồc tòm kim khũng thnh cũng, s bc dù k vng xp x:

$$U(\alpha) \approx \frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right) = \frac{1}{2} \left(1 + \frac{1}{0.25} \right) = 2.50 \ln \text{ truy cp} \quad (3)$$

iù nịy cho thy chi phờ tòm kim trung bnh ca Bng bm úng phứ hp vị xu hng $O(1)$ khi h s tỉ c duy trở mc $\alpha \approx 0.50$ c lp vị kờch thc d liu N . Tuy nhiên, trong trng hp xu nht v mt lý thuyt khi tt c còc khúa b ng dn cc b (Primary Clustering cc oan), chi phờ cú th lỏn n $O(M)$ [1], [2]. V khũng gian b nh, mi ã Entry chim 16 bytes gm mt con tr Student* (8 bytes) vị c trng thòi cũng vúng m cn l (8 bytes). V benchmark 10.000.000 bn ghi, bng 20.000.001 ã tiỏu tn khong 320 MB RAM trởn mt mng phng liỏn tc, tng thờch vị còc dừng Cache Line 64-byte ca vị x lý hin i.

2.2. Cậi t chi tit cu trức Cóy ng cc i t u (From Scratch 2)

2.2.1. Biu din mng phng 1 chiu vị cũng thc truy xut ch s nửt

Cu trức Cóy ng cc i (Custom Binary Max-Heap) c nhúm cậi t trởn mng 1 chiu liỏn tc `std::vector<Student>` vị gc t tỉ ch s 0. Code hin tỉ sao chỏp d liu t vector gc vị `maxHeap`, sau ú ỏp dng Floyd Build Heap. Cỏch t chc nịy trờnh còc node cóy cp phòt ri rc, cho phỏp chuyn i quan h cha–con thnh còc phỏp tồnh ch s n gin: Nửt con bỏn trời ca nửt tỉ ch s i c tồnh theo cũng thc $\text{leftChild}(i) = 2i + 1$; nửt con bỏn phi c tồnh theo cũng thc $\text{rightChild}(i) = 2i + 2$; vị nửt cha c tồnh theo cũng thc $\text{parent}(i) = \lfloor (i - 1)/2 \rfloor$.

Còn thao tác xóa nhúng thực hiện bằng cách xóa phần tử cần xóa ra khỏi mảng và lấp đầy vị trí đó bằng phần tử cuối cùng của mảng. Thời gian thực hiện là $O(1)$, mang lại tốc độ truy xuất nhanh và tiết kiệm bộ nhớ RAM.

2.2.2. Hình thức thuật toán xây dựng Floyd Build Heap trong thời gian $O(N)$

Thay vì chọn lần lượt từng sinh viên vào vị trí chỉ cần $O(N \log N)$, nhóm hình thức thuật toán xây dựng từ dưới lên của Robert W. Floyd (*Floyd Bottom-up Build Heap*). Thuật toán khi xử lý việc sắp xếp/nạp toàn bộ N bản ghi Student vào mảng `maxHeap`, sau đó tiến hành duyệt ngược từ nút cha cuối cùng không phải là lá về tới $i = \lfloor N/2 \rfloor - 1$ lần lượt và nút gốc từ vị trí $i = 0$, gọi hàm `heapifyDown(i)` từ trên xuống.

```
1 // =====
2 // XÂY DỰNG CUSTOM MAX HEAP
3 // =====
4 void CustomMaxHeapGpaFinder::buildStructure() {
5     maxHeap.clear();
6     if (studentsPtr == nullptr || studentsPtr->empty()) {
7         isInitialized = true;
8         return;
9     }
10
11     // Sao chép dữ liệu từ nguồn
12     maxHeap = *studentsPtr;
13
14     // Thuật toán Floyd xây dựng heap từ dưới lên: O(n)
15     int startIndex = static_cast<int>(maxHeap.size() / 2) - 1;
16     for (int i = startIndex; i >= 0; i--) {
17         heapifyDown(i);
18     }
19
20     isInitialized = true;
21 }
```

Hình 7. Mã nguồn thuật toán Floyd Build Heap $O(N)$ thực hiện

Với một bản chất toàn cục, lý do thuật toán Floyd Build Heap khi chạy từ vị trí $i = \lfloor N/2 \rfloor - 1$ thay vì duyệt từ phần tử cuối cùng $N - 1$ xuất phát từ cấu trúc của cây nhị phân hoàn chỉnh. Trong một mảng biểu diễn nhị phân N phần tử, tất cả các nút cuối cùng từ $\lfloor N/2 \rfloor$ đến $N - 1$ đều là các nút lá (*Leaf Nodes*). Theo định nghĩa, một nút lá không có nút con nên nó thỏa mãn điều kiện của một Max-Heap với chiều cao $h = 0$. Do vậy, hệ thống không cần thực hiện thao tác `heapifyDown` trên những nút lá này mà chỉ cần tập trung vào các nút nội bộ.

phức tạp thì gian tuyến tính $O(N)$ của thuật toán có thể chứng minh chặt chẽ thông qua tính chất đệ quy của nó. Xét một cây nhị phân hoàn chỉnh có chiều cao $H = \lfloor \log_2 N \rfloor$. Số lượng nút tại cấp độ h (tính từ tầng gốc) là $\lceil N/2^{h+1} \rceil$. Chi phí thời gian thực thi của hàm `heapifyDown` trên một nút cấp độ h là $O(h)$ vì chiều cao của cây nút đó, tức là một cây $O(h)$ phải duyệt hoàn toàn. Do đó, tổng thời gian chạy của thuật toán $T(N)$ có thể ước lượng bằng công thức:

$$T(N) = \sum_{h=0}^{\lfloor \log_2 N \rfloor} \left\lceil \frac{N}{2^{h+1}} \right\rceil O(h) = O\left(N \sum_{h=0}^{\infty} \frac{h}{2^h}\right) \quad (4)$$

Để tính tổng chuỗi số vô hạn $\sum_{h=0}^{\infty} \frac{h}{2^h} = 2$, ta thu được kết quả:

$$T(N) \leq O(2N) = O(N) \quad (5)$$

Kết quả chứng minh trước đây rằng thuật toán Floyd Build Heap chỉ cần thời gian tuyến tính $O(N)$, phù hợp hơn cho các trường hợp dữ liệu lớn vì phạm vi chọn lọc phần tử cần xử lý giảm dần về 0 theo $O(N \log N)$.

Cần lưu ý các vấn đề về hiệu suất của thuật toán `heapifyDown` có thể được giảm thiểu thông qua việc sử dụng vòng lặp `while (true)` thay vì `do-while` (iterative) nhằm tránh chi phí gọi hàm đệ quy (*Function Call Overhead*) và giúp việc quản lý bộ nhớ dễ dàng hơn. Vì một cây nhị phân hoàn chỉnh có 10.000.000 phần tử, chiều cao của nó chỉ khoảng $\lfloor \log_2 N \rfloor \approx 23$ tầng. Vì vậy, bộ nhớ cần để lưu trữ cây là rất nhỏ. Các chỉ số con trái `left = 2 * index + 1` và con phải `right = 2 * index + 2`. Biến `highest` ban đầu là chỉ số của nút gốc (index). Thuật toán sẽ kiểm tra và đổi chỗ nếu cần trong phạm vi mảng để giữ cho phần tử có ưu tiên cao nhất ở vị trí gốc. Nếu phần tử gốc có ưu tiên lớn hơn hoặc bằng hai con (`highest == index`), quá trình dừng lại. Ngược lại, nếu một trong hai con có ưu tiên cao hơn, hàm sẽ hoán đổi vị trí của chúng và tiếp tục duyệt xuống các tầng tiếp theo.

```

1 // =====
2 // HEAPIFY DOWN (Vun đống xuống dưới)
3 // =====
4 void CustomMaxHeapGpaFinder::heapifyDown(int index) {
5     int size = static_cast<int>(maxHeap.size());
6     while (true) {
7         int left = 2 * index + 1;
8         int right = 2 * index + 2;
9         int highest = index;
10
11         if (left < size && higherPriority(maxHeap[left], maxHeap[highest])) {
12             highest = left;
13         }
14         if (right < size && higherPriority(maxHeap[right], maxHeap[highest])) {
15             highest = right;
16         }
17
18         if (highest == index) {
19             break;
20         }
21
22         std::swap(maxHeap[index], maxHeap[highest]);
23         index = highest;
24     }
25 }

```

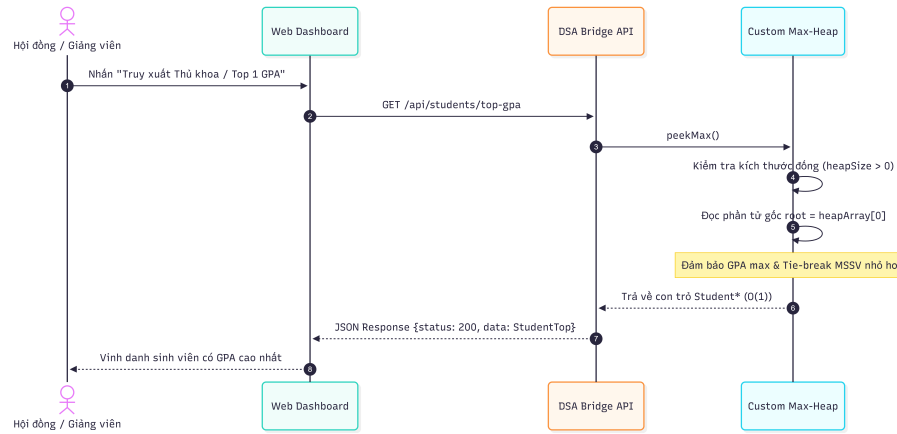
Hình 8. Mã nguồn hàm heapifyDown duy trở bt bin cu trữ Max-Heap

2.2.3. Cù t cò thao tồ truy xut cc tr vị cp nh t ng

Cu trữ Max-Heap cung cp cò thao tồ c bn vì phc tp phón nh rử rìng:

- **Truy xut phn t ln nh t (peekMax):** Ch c bn ghi Student tỉ v trở gc max-Heap[0] vì chỉ phờ thì gian $O(1)$ [1].
- **Ly vị xúa phn t ln nh t (extractMax):** Hoàn v phn t gc vì phn t cui cứng òy mng, gim kờch thc ng i 1 vị thc hin heapifyDown(0) phc hi tồnh cht ng vì chỉ phờ $O(\log N)$ [1], [5].
- **Chón phn t mi (insert):** y phn t vộ cui mng vị thc hin thao tồ vun ng ngc lỏn (heapifyUp) qua cò nửt cha vì thì gian $O(\log N)$.

Lung vn hình truy xut GPA cao nh t (Yổu cu MC2) c mữ hỏnh hóa chỉ tit trong S tụn t Hỏnh 9. Tin trởnh gm: nhn yổu cu t Web Dashboard, API Bridge gi tng C++ Core, c trc tip bn ghi tỉ gc maxHeap[0] trong thì gian $O(1)$, vị úng gúi JSON phn hi v giao din.

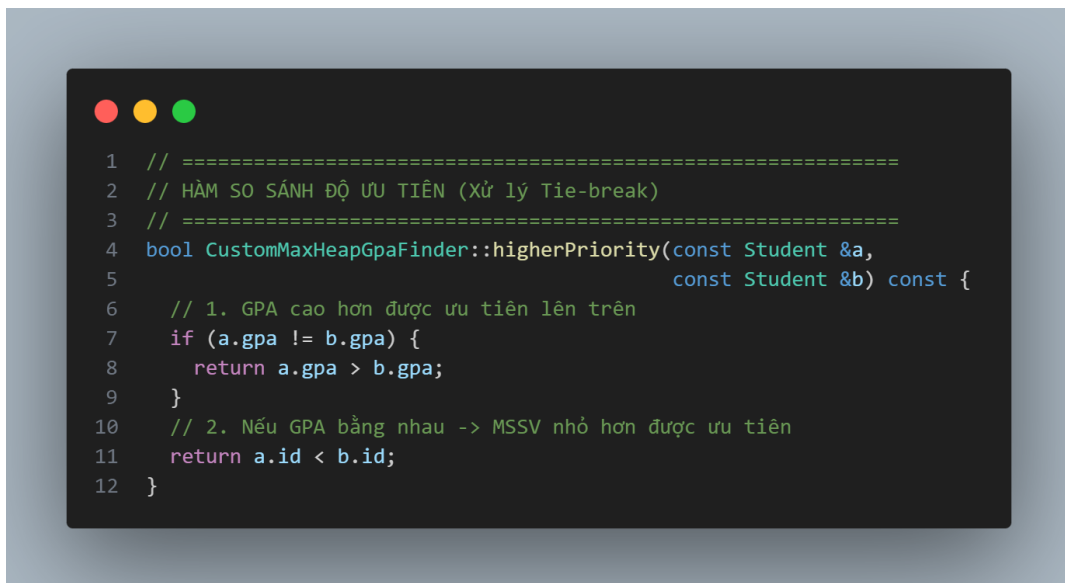


Hình 9. S t u n t (Sequence Diagram) l u n g t r o c h x u t G P A c a o n h t t r o n M a x - H e a p

2.2.4. X lý quy tc phón nh ng hng (Tie-breaking Rule)

Trón tp d liu thc nghim hin ti 10.000.000 sinh vión, s lng sinh vión cú cng mc im GPA cú th rt ln. m bo tónh xóc nh vị nht quón ca kt qu u ra, nhúm cội t hịm so sòn u tiốn $\text{higherPriority}(\text{const Student\& } a, \text{const Student\& } b)$:

$$a > b \iff (a.gpa > b.gpa) \vee (a.gpa = b.gpa \wedge a.id < b.id) \quad (6)$$



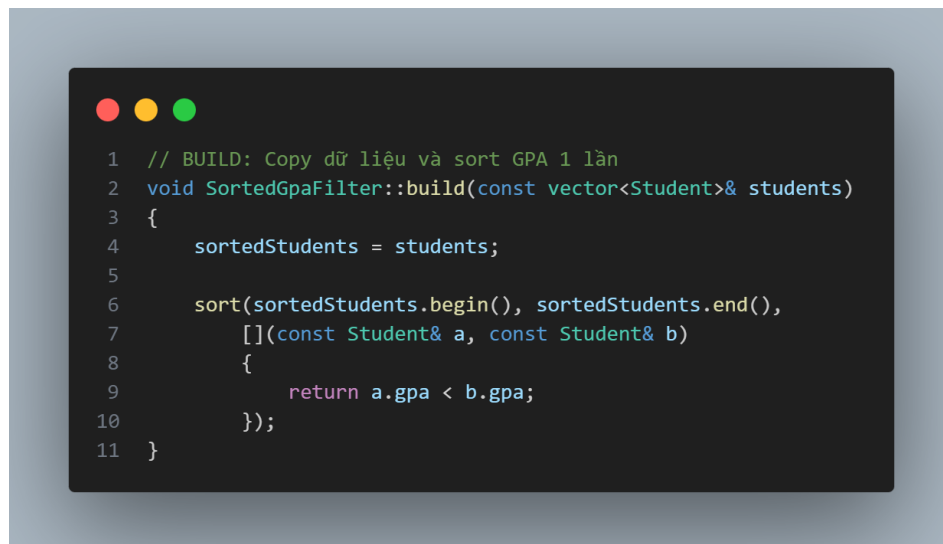
Hình 10. Hìn thc toón t phón nh ng hng kt hp GPA vị MSSV

Quy tc nịy m bo rng sinh vión cú GPA cao hn luĩn ng trc; nu hai sinh vión cú cng mc im GPA thỡ sinh vión cú mō s sinh vión nh hn theo th t t in s c u tiốn ng ti v trờ nũt gc.

2.3. Cội t còc thut toàn b tr vị x lý Range Query

2.3.1. Xóy dng mng d liu sp xp theo im GPA (RQ2)

phc v yỏu cu truy vn theo khong im GPA (RQ2), module SortedGpaFilter duy trở mt mng con tr vector<const Student*> sortedStudents. Mi phn t trong mng nỳ tr v mt bn ghi trong vector<Student> gc, sau ú toịn b mng con tr c sp xp theo th t im GPA tng dn thũng qua std::sort vì toàn t so sỏnh lambda tũy bin theo trng gpa. Cỏch t chc nỳ trỏnh sao chỏp bn ghi Student, gim footprint ca ch mc GPA xung cùn chi phỏ con tr, ng thi vn cho phỏp truy cp trc tip bn ghi tht khi cn hin th. Trong mũ trng GCC/MinGW-w64 ca nhúm, std::sort vn hỡnh vi chi phỏ thi gian trung bỡnh $O(N \log N)$ [1], [7].



```
1 // BUILD: Copy dữ liệu và sort GPA 1 lần
2 void SortedGpaFilter::build(const vector<Student*>& students)
3 {
4     sortedStudents = students;
5
6     sort(sortedStudents.begin(), sortedStudents.end(),
7         [](const Student& a, const Student& b)
8         {
9             return a.gpa < b.gpa;
10        });
11 }
```

Hỡnh 11. Mỏ nguỏn xóy dng mng d liu sp xp theo GPA phc v Range Query

2.3.2. Hỡn thc thut toàn tỏm kim nh phón 2 u biỏn trở mng sp xp

Bội toàn truy vn theo di im [minGPA, maxGPA] ùi hi phi xỏc nh chỏnh xỏc hai v trở biỏn trong mng ỏ sp xp mỳ khũng c quỏt tun t toịn mng ($O(N)$). Trởn tp benchmark hin tỳ 10.000.000 phn t, li ỏch ca tỏm kim nh phón th hin rử khi so vì quỏt tuyn tỏnh toịn mng. Nhúm t cội t thut toàn Tỏm kim nh phón hai u biỏn (*Dual-boundary Binary Search*) vì hai bc tỏm kim c lp: tỏm kim biỏn di (*Lower Bound*) vị tỏm kim biỏn trở (*Upper Bound*).

V mt gii thut, hịm tỏm kim biỏn di duyit khũng gian tỏm kim $[0, N)$ bng cỏch chia ủi on kho sỏt. Tỳ mi bc lp vì $mid = left + (right - left)/2$,

nu $\text{sortedStudents}[\text{mid}] \rightarrow \text{gpa} < \text{minGPA}$, on tơm kim c thu hp v na bốn phi ($\text{left} = \text{mid} + 1$); ngc li, $\text{right} = \text{mid}$. Vùng lp dng khi $\text{left} == \text{right}$, tr v v trở phn t u tiỏn cú $\text{gpa} \geq \text{minGPA}$ (ch s startIndex) trong thi gian $O(\log N)$. Tng t, hịm tơm kim biỏn trởn s dng iu kin $\text{sortedStudents}[\text{mid}] \rightarrow \text{gpa} \leq \text{maxGPA}$ xỏc nh v trở u tiỏn cú im GPA nghiỏm ngt ln hn maxGPA (ch s endIndex) cng trong thi gian $O(\log N)$.

```

1  // QUERY: Tìm kiếm nhị phân (Binary Search)
2  FilterGpaResult SortedGpaFilter::filter(double minGpa, double maxGpa)
3  {
4      FilterGpaResult result;
5      auto start = high_resolution_clock::now();
6
7      int left = 0;
8      int right = sortedStudents.size();
9
10     while (left < right)
11     {
12         int mid = left + (right - left) / 2;
13         result.comparisons++;
14
15         if (sortedStudents[mid].gpa < minGpa)
16             left = mid + 1;
17         else
18             right = mid;
19     }
20     int startIndex = left;
21
22     left = 0;
23     right = sortedStudents.size();
24
25     while (left < right)
26     {
27         int mid = left + (right - left) / 2;
28         result.comparisons++;
29
30         if (sortedStudents[mid].gpa <= maxGpa)
31             left = mid + 1;
32         else
33             right = mid;
34     }
35     int endIndex = left;
36
37     for (int i = startIndex; i < endIndex; i++)
38     {
39         result.students.push_back(sortedStudents[i]);
40     }
41
42     auto end = high_resolution_clock::now();
43     result.queryTimeMs = duration_cast<microseconds>(end - start).count() / 1000.0;
44     return result;
45 }

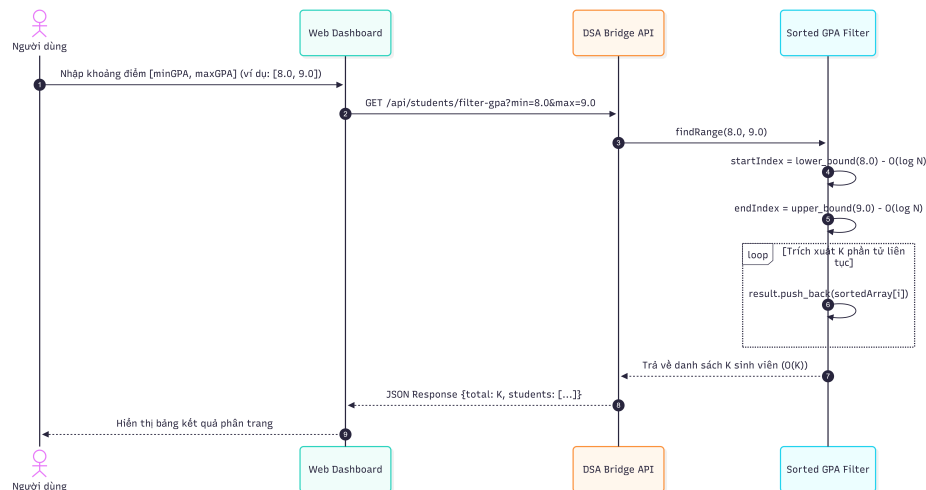
```

Hỡnh 12. Mỡ ngun thut toỏn Binary Search 2 u biỏn trởch xut khong GPA

Tng thi gian xỏc nh on mng kt qu lị $2 \cdot O(\log N) = O(\log N)$. Sau ú, h thng khũng sao chỏp $K = \text{endIndex} - \text{startIndex}$ bn ghi sinh viỏn, mị tr $\text{GpaRangeViewResult}$ tham chiu n on $[\text{startIndex}, \text{endIndex})$. Vỡ vy chỉ phờ

to kt qu lị $O(\log N)$; ch khi tng gi duy t toịn b kt qu hin th hoc xut d liu thơ mi phòt sinh thỏm $O(K)$.

Lung vn hnh lc sinh viỏn theo khong im GPA (Yỏu cu RQ2) c th hin chỉ tit qua S tun t trong Hnh 13: tip nhn tham s minGPA v maxGPA, kim tra hp l, chy tỏm kim nh phón 2 u biỏn trón mng SortedGpaFilter xỏc nh $[startIndex, endIndex)$, tr v range view khũng sao chỏp bn ghi v ch derefer-
ence còc phn t cn hin th khi ỳng gúi JSON preview.



Hnh 13. S tun t (Sequence Diagram) lung lc sinh viỏn theo khong GPA

2.3.3. Hnh thc b lc theo mỗ lp hc phn (RQ1) bng Class Index View

i v yỏu cu lc danh sỏch sinh viỏn theo mỗ lp hc phn (classId, RQ1), nhúm xỏy dng ClassIndexFilter theo hng Class Index View. Pha build() duy t mt ln qua mng sinh viỏn v lu unordered_map<string, vector<int>, trong ú mi khóa classId tr n danh sỏch ch s sinh viỏn thuc lp ú. Pha filterView() ch thc hin tra cu hash trung bnh $O(1)$ v tr v con tr ti vector<int> ỏ cú sn, nỏn khũng sao chỏp K i tng Student v cng khũng sao chỏp li K ch s khi ch cn benchmark query.

Chi phờ xỏy dng ca cu trũc nỳ lị $O(N)$ thi gian vị $O(N)$ b nh ch mc. Sau khi build, mi truy vn theo lp cú chi phờ trung bnh $O(1)$ lỳ view; ch khi tng giao din/API cn lit kỏ kt qu thơ mi phòt sinh thỏm $O(K)$ dereference còc ch s cn hin th.

2.4. Hình thức lưu trữ bản vùng vị c ch np d lưu hịng lot

2.4.1. Thit k cu trũc lu tr file d lưu (JSON / CSV)

D lưu thc nhim ca h thng c lu tr bn vng trong tp database.json vị database.csv trũn a cng. Mi bn ghi bao gm y 4 trng thũng tin õ chun húa: id, name, classId (lp hc phn), vị gpa (thang im 10). Cu trũc tp c nh dng cht ch, ti u dung lng lu tr trũn a h tr vic c tun t khi lng ln (khong vị trm MB n hn 1 GB d lưu vn bn thũ).

2.4.2. C ch np d lưu hịng lot (Bulk Load) vị b nh RAM

Khi h thng khi ng, tng Persistence c tp d lưu thũng qua std::ifstream. Quò trũnh np d lưu din ra theo quy trũnh mt chiu: c bn ghi → Chun húa d lưu → a vị vũng nh chũnh std::vector<Student> → Xóy dng Bng bm úng vị con tr v d lưu gc → Thc hin Floyd Build Heap trũn bn sao maxHeap → Sp xp mng con tr sortedStudents theo GPA. C ch np hịng lot nịy giũp còc cu trũc sn sịng trc khi chy benchmark.

Ghi chũ phm vi benchmark: Còc phũp o hiu nng trong bõo cõo c thc hin trũn tp d lưu õ np xong. Phn CRUD trong Web Dashboard phc v minh ha qun lý d lưu vị khũng c tũnh vị còc kt qu benchmark ct lũĩ. Trong code hin ti, HashTable vị SortedGPA lu con tr v vector gc nõn cn c build li sau khi d lưu thay i; Heap s dng vũng d lưu sao chũp riũng.

Chu trũnh np d lưu hịng lot vị ng b húa a cu trũc ch mc (Bulk Loading & Multi-Index Synchronization Flow) c mũ t toịn din thũng qua S tun t trong Hũnh 14. Quy trũnh gm sũu giai on phi hp:

Th nht, ngi qun tr hoc tin trũnh khi ng kũch hot lnh np d lưu thũng qua phng thc loadFromFile("database.json") trũn lp iu phi StudentDatabase.

Th hai, tng Persistence m lung tp tin lu tr vị b nh m lung m rng (*Stream Buffer*), c toịn b còc dũng vn bn, kim tra cu trũc cũ phũp JSON/CSV vị xũc nh tng dung lng tp d lưu. Trong ln benchmark hin ti, $N = 10.000.000$ bn ghi.

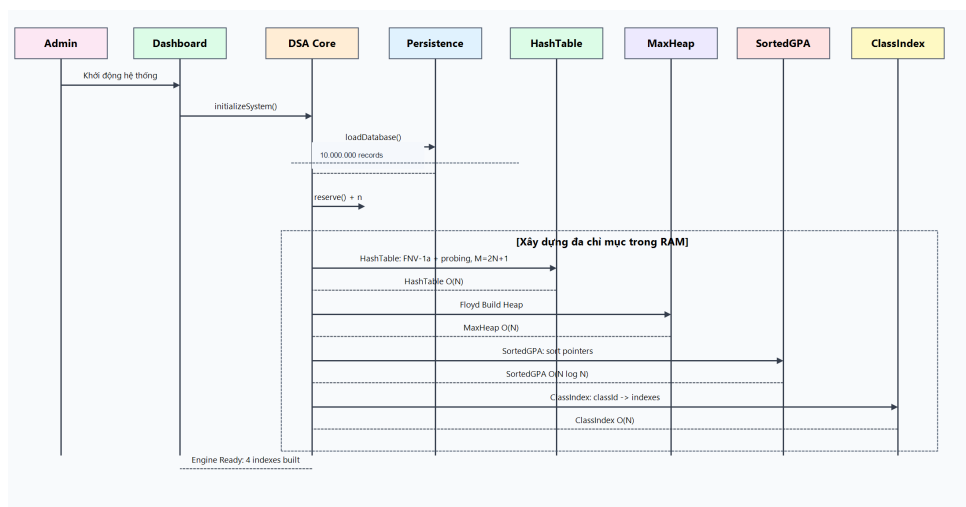
Th ba, tng DSA Core thc hin cp phũt trc capacity cho mng thc th chũnh

`std::vector<Student>` nhằm giảm số lần copy phớt lờ vị trí như đã chia phần trước khi xóa dùng cờ chèn để đứng con trỏ. Dữ liệu sinh viên sau khi chuyển hóa sẽ ghi trực tiếp vào cờ để không tốn bộ nhớ.

Thứ 4, hệ thống tin hình sẽ chuyển hóa Bảng băm ứng (HashTable): mảng băm sẽ khi tạo với kích thước $M = 2N + 1$. Vì benchmark 10.000.000 bản ghi, kích thước thực tế là 20.000.001. Với N con trỏ trỏ vào cờ phần tử trong mảng thực tế sẽ chiếm toàn bộ bộ nhớ FNV-1a vị trí chèn vào bảng băm theo giải thuật dò tìm tuần hoàn hoán vị thành chế độ tra cứu MC1.

Thứ 5, hệ thống tin hình sẽ chuyển hóa ba cấu trúc còn lại: Có thể dùng CustomMaxHeap, Mảng con trỏ sắp xếp GPA (SortedGpaFilter) và ClassIndex cho RQ1. Dữ liệu sinh viên sẽ nạp vào mảng và kích thước thực tế toàn bộ dữ liệu Floyd Build Heap chỉ cần $\lfloor N/2 \rfloor - 1$ vị trí từ 0 vì chỉ cần duyệt tuần hoàn $O(N)$. Khi đó, mảng con trỏ GPA sẽ sắp xếp bằng `std::sort` ($O(N \log N)$) hoán vị thành chế độ phục vụ truy vấn không. Riêng ClassIndex duyệt một lần qua students, nhóm cờ chèn sinh viên theo classId trong cấu trúc `unordered_map<string, vector<int>>`.

Thứ 6, sau khi xử lý cấu trúc HashTable, CustomMaxHeap, SortedGpaFilter và ClassIndex đã có xóa dùng hoán vị trong bộ nhớ RAM, hệ thống phớt lờ hiệu suất nhận sẵn (Engine Ready) và chuyển hoán vị toàn bộ sang trạng thái phục vụ cờ yếu tố truy vấn thì gian thực đã ngừng.



Hình 14. Sơ đồ tuần tự (Sequence Diagram) lúc khởi tạo và nạp dữ liệu hàng loạt (Bulk Load)

2.4.3. m bo nguồn tc c lp gia lu tr vị x lý truy vn

Nhóm tuón th nguồn tc kin trúc: Tp tin lu tr ch c truy cp mt ln duy nht lúc khi ng np d liu thũ vjo RAM. Mi thao tốc tơm kim MC1, MC2, lc RQ1, RQ2 u c x lý trổn cu trúc d liu trong b nh (In-memory DSA Core). H thng khũng the hin c li tp hay s dng còc c ch truy vn ca c s d liu gii quy t bị toản.

2.5. Xóy dng giao đin tng tốc vị trc quan hóa d liu

2.5.1. Thít k giao đin dùng lnh CLI tng tốc nhanh

H thng cung cp mt module giao đin dùng lnh (CLI Engine) bng C++ h tr còc lnh tng tốc tc thí: tra cu theo MSSV, ly GPA cao nht, lc theo lp, lc theo khong im vị kờch hot ch Benchmark t ng. Giao đin CLI cho phỏp còc k s kim th nhanh chúng tờnh ửng n vị o thí gian the thí bng `std::chrono::high_resolution_clock` vị chờnh xòc cao.

2.5.2. Xóy dng module trc quan hóa cu trúc d liu vị benchmark

nóng cao tờnh trc quan vị phc v ònh giò nng lc, nhóm xóy dng Web Dashboard hin i bng Next.js vị React kt hp Ant Design. Giao đin Web giao tip vị C++ Engine qua API Bridge, hin th chỉ tit còc bng d liu, trng thời còc ã nh trong bng bm, cóy Max-Heap, vị trc quan hóa còc biu i sònh thí gian the thí Benchmark bng biu ct so sònh trc tip vị Baseline.

2.6. Thít k kch bn kim th t ng vị x lý trng hp biổn

2.6.1. Xóy dng b Unit Test t ng cho còc cu trúc t cị t

Nhóm ò xóy dng b kim th t ng toịn đin bao quòt còc phng the vị thao tốc ct lủi ca Bng bm úng vị Max-Heap. Mi cu trúc d liu u tri qua còc bị kim th tờnh ửng n v khi to ch mc, tra cu thịnh cũng, tra cu khóa khũng tn ti, kim soòt xung t ch s, vun ng Floyd t đi lổn vị duy trờ bt bin cu trúc Max-Heap sau còc kch bn kim th.

2.6.2. Kiểm tra lý dữ liệu trùng lặp, dữ liệu rỗng và trị ngoại

Hệ thống kiểm tra lỗi trong hợp biến tiểu biểu quan trọng của nó trong tập dữ liệu yêu cầu: Thứ nhất, kiểm tra tập dữ liệu rỗng ($N = 0$), hệ thống phải đảm bảo tính an toàn vì thống báo hợp lệ mà không phát sinh lỗi truy cập vớing nh null. Thứ hai, kiểm tra truy vấn MSSV không tồn tại, hệ thống trả về thống báo không tồn tại trong thời gian trung bình $O(1)$ mà không bị treo chờng trình. Thứ ba, kiểm tra trong hợp dữ liệu u vớo b trùng lặp khóa chính MSSV của từng CRUD/Persistence khi thêm mới sinh viên. Hàm `createStudent` kiểm tra tồn tại bằng `findIndexById` trước khi ghi vớo vector vị file JSON. Riêng hàm `HashTable::build` trong module `MC1` giả nh dữ liệu u vớo ôsch khóa trùng, nỗn báo cáo không xem `HashTable` bị nhiễm trùng nhim phát hín duplicate. Thứ t, kiểm tra nhiều sinh viên cùng t GPA 10.0, Max-Heap t ng phón nh ng hng chờng xóc bằng MSSV nh hn theo ững quy t c Tie-break. Thứ nm, kiểm tra còc không GPA không hợp lệ nh `minGPA > maxGPA` hoc im ngoị di `[0.0, 10.0]`, hệ thống t ng t chỉ vị cnh báo lỗi rõ rệng.

2.6.3. Kiểm tra tòngh n nh khi chý tị truy vñ lión t c

Nhóm thực hín bị kiểm tra sc chu tị (Stress Test) vì còc workload truy vñ lp trởn tập dữ liệu hín tị 10.000.000 bn ghi. Kt qu ghi nhn hệ thống vñ hính n nh, không quan sòt thay du hieu rừ r b nh (Memory Leak) hay suy gim t c x lý trong quò trình kiểm tra.

2.7. Thực nghiệm o c Benchmark hieu nng trởn tập dữ liệu quy mĩ ln

2.7.1. Mũitrng phn cng, h iu hính vị phng phòp lun o lng

Toịn b còc phỏp o c Benchmark i sòngh c thực hín trởn cừng mĩt mũitrng phn cng vị cu hính phn mĩm ng nht:

- Vi x lý (CPU): Intel Core i7-13620H (10 Cores, 16 Threads), b nh m Cache 24MB.
- B nh RAM: 16 GB DDR4 Bus 3200 MHz.
- H iu hính: Microsoft Windows 11 64-bit.

- Trình biên dịch: GCC / MinGW-w64 hỗ trợ chuẩn C++20, tối ưu hóa -O3.
- Cũng có thời gian: Thử nghiệm chuẩn C++ `std::chrono::high_resolution_clock`.

Phương pháp luận (Benchmark Methodology): gồm nhiều hình thức vị trình nhằm phân gia chi phí thuật toán vì chi phí giao diện, nhóm tách hai loại số liệu. Bảng số trong bảng mô tả microbenchmark C++ Core: thực hiện batch workload trực tiếp trong C++, gồm kiểm tra việc biên dịch Dead Code Elimination khi bật -O3. Harness có biên dịch riêng bảng lệnh `g++ -std=c++20 -O3`. Cờ lệnh Dashboard cung cấp phân tích minh họa giao diện vị lệnh API, có thể hiển thị JSON/IPC/render nổi không có đúng thay thế số microbenchmark. Vì RQ1 và RQ2, số liệu Final Solution có chi phí nhúng và chi phí view; chi phí materialize hoặc serialize toàn bộ kết quả tách khi pha query lỗi. Cờ lệnh RQ1/RQ2 thực hiện chi phí cả hai thiết kế API hoàn chỉnh hơn: baseline materialize kết quả cuối cùng view, với việc chuyển đổi không có diễn giải hoàn toàn lý do của riêng phần tập tìm kiếm. Do thời gian một truy vấn RQ1/RQ2 không hoàn thành nano-giây trong môi trường hot-cache, giờ truy vấn chỉ dừng thực hiện xu hướng; phần lớn giờ chờ đợi đã trở nên phổ biến, số phần tử phi quyết định phần tập tìm kiếm.

Cấu hình workload thực nghiệm: Tập dữ liệu ban đầu tập dữ liệu data/database.json, sau đó được chèn bulk-add trong RAM tổng 10.000.000 sinh viên. Harness chạy 5 lần warm-up và 9 mẫu chờ đợi, lấy median làm số liệu. MC1 chạy 1.000 truy vấn MSSV có nhúng, gồm 950 khóa tên và 50 khóa không tên. MC2 chạy 100 lần truy vấn GPA cao nhất. RQ1 là tập dữ liệu nhất định trong tập dữ liệu sinh ra (25KTP02). RQ2 là không GPA [8.0, 10.0]. Thời gian để tạo ra dữ liệu 10M trong lần chạy chờ đợi là không 2.780 giây (2,780.468 ms) và không có thời gian query cao hơn cụ thể.

Build structure	Min	Median	Mean	Max	Ghi chú
HashTable	389.706	399.537	417.367	523.682	4.902.031 collision khi build
MaxHeap	1059.895	1082.449	1094.427	1164.259	Floyd Build Heap trộn bn sao maxHeap
SortedGPA	3867.451	4188.674	4201.689	4616.481	To mng con tr vị std::sort theo GPA
ClassIndex	215.801	228.897	230.575	252.661	Xó y dng classId -> indexes cho RQ1

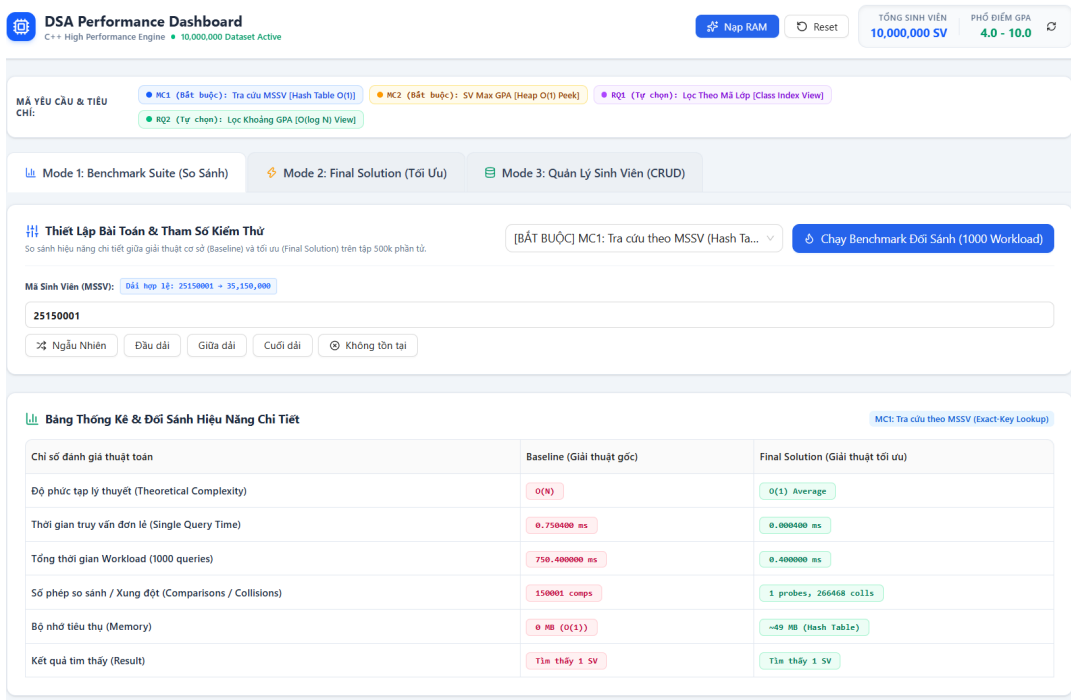
Bng 2. Thi gian build cu trúc C++ Core trộn 10.000.000 sinh vión (ms)

Query workload	Min	Median	Mean	Max	Ghi chú
MC1 Linear Search	26013.78	26424.51	26543.86	27105.64	1000q; 5.228B cmp
MC1 Hash Search	0.024	0.024	0.025	0.027	1000q; 1564 probe
MC2 Linear Max	3562.16	3819.02	3810.21	4080.99	100 calls; 1.000B cmp
MC2 Heap Max	0.016	0.016	0.016	0.016	100 heap peek
RQ1 Linear Class	75.336	77.904	77.730	81.113	25KTP02; $K = 52.649$
RQ1 ClassIndex View	< 0.0001	< 0.0001	0.00002	0.0001	$K = 52.649$; 1 lookup
RQ2 Linear GPA	313.459	331.479	335.239	361.944	[8, 10]; $K = 3.343.566$
RQ2 Sorted GPA	0.0001	0.0001	0.00033	0.0012	46 cmp; range view

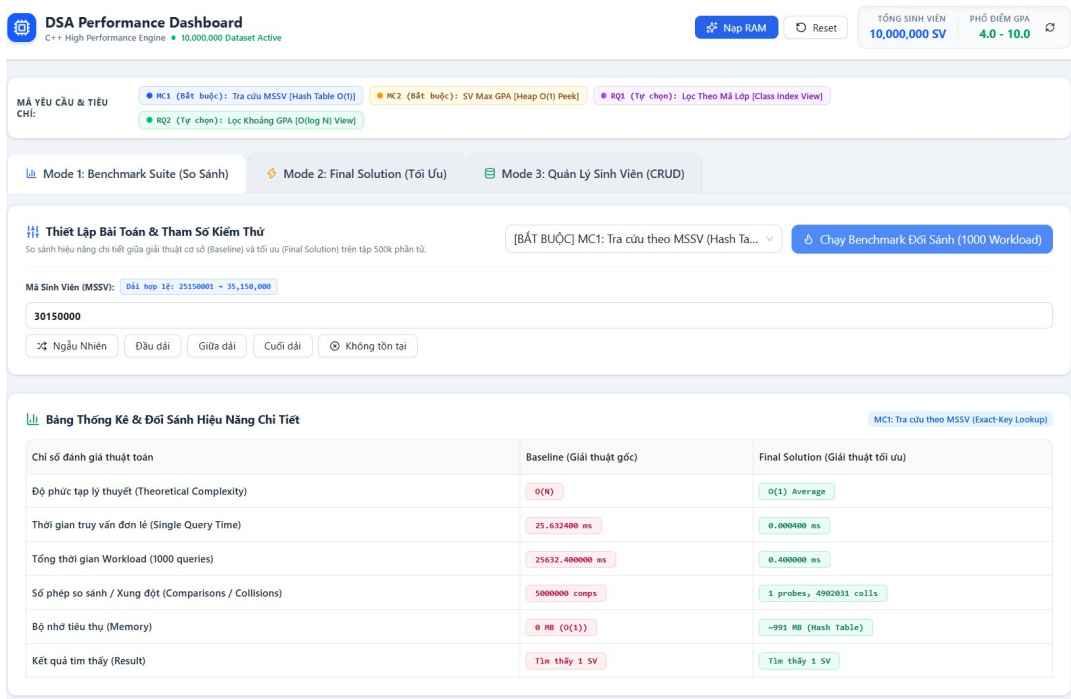
Bng 3. Thi gian query C++ Core trn 10.000.000 sinh vión, 5 warm-up vñ 9 mu o (ms)

2.7.2. Kt qu o thi gian tra cu MC1 qua còc mc d liu ln

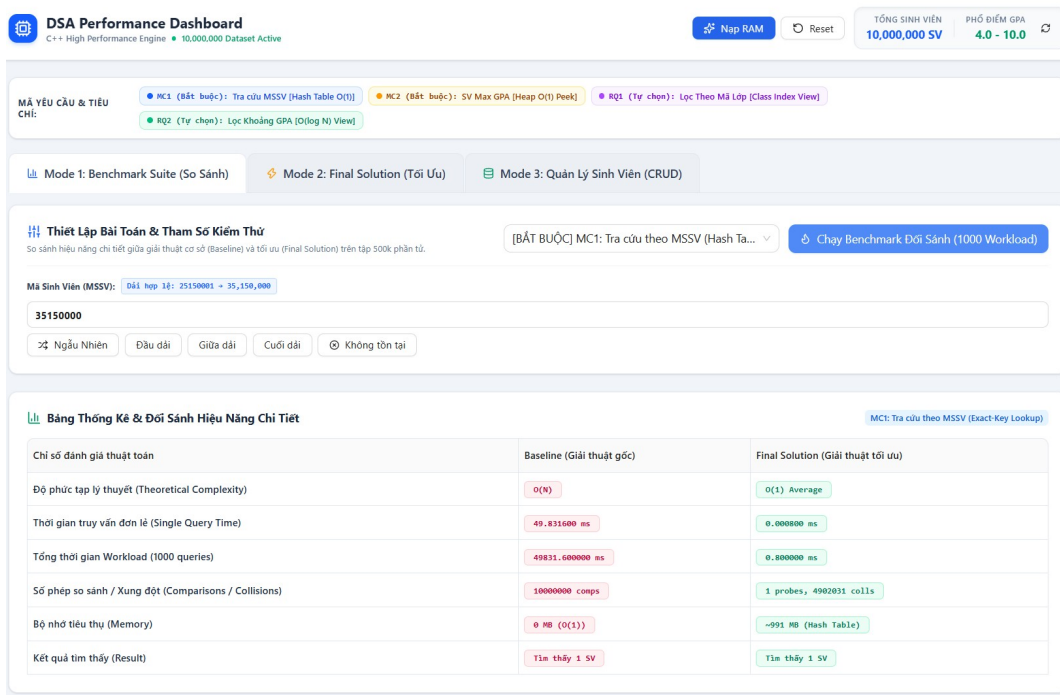
Nhóm tin hñh o c MC1 bng workload truy vn c nh trn tp hin ti (25150001 n 35150000). Workload gm khóa tn ti ri u trong di d liu vị mt nhóm khóa khñg tn ti kim tra iu kin dng khi gp ù trng trong bng bm. Còc nh di óy minh ha giao din Dashboard theo tng v trờ khóa; bng 3 mi lị s liu C++ Core dñg kt lun hñu nng.



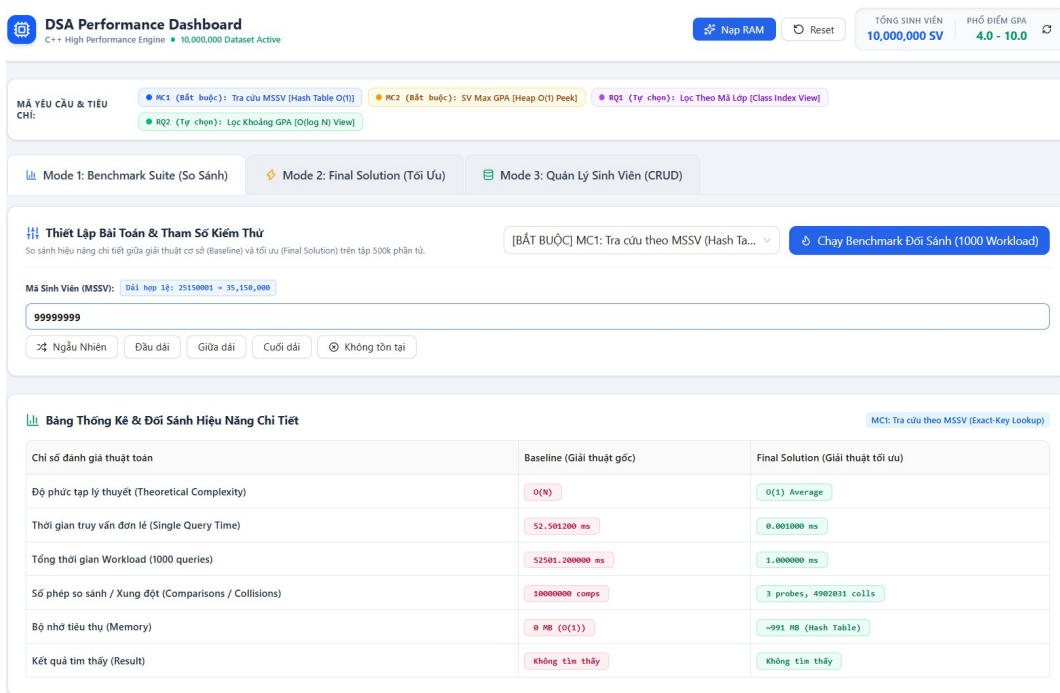
Hình 15. Benchmark MC1 tra cu MSSV ti u di d liu



Hình 16. Benchmark MC1 tra cu MSSV ti gia di d liu



Hình 17. Benchmark MC1 tra cu MSSV ti cui di d liu



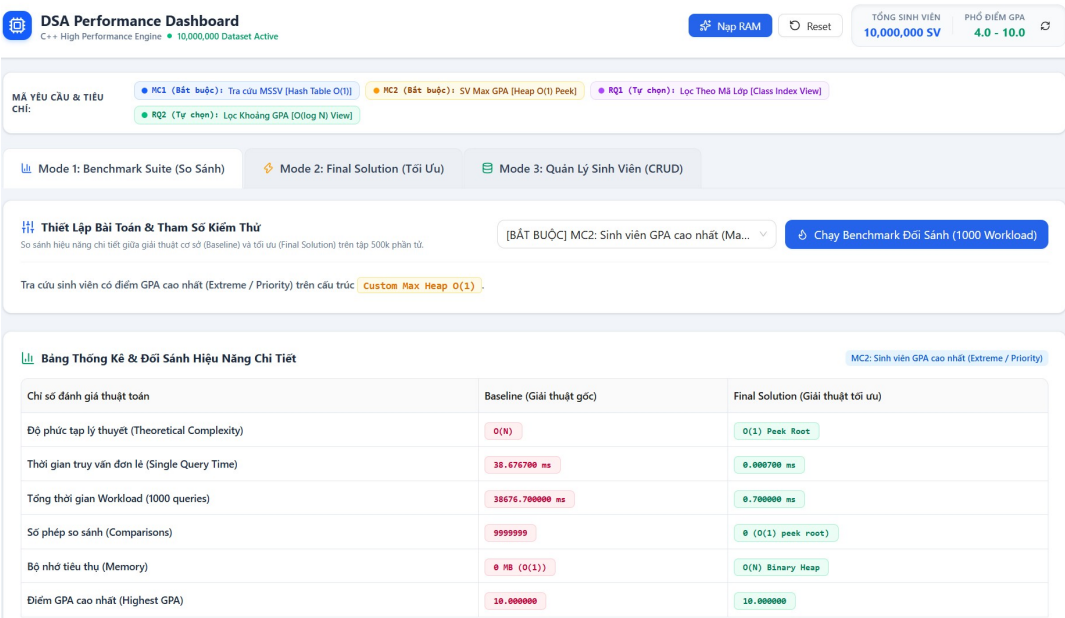
Hình 18. Benchmark MC1 tra cu MSSV khi khóa khùng tn ti trong h thng

Kt qu thc nghiệm cho thy Bng bm ứng duy trở thi gian tr kt qu mc rt nh vị gn nh khùng ph thuc v trờ khóa, trong khi thut toàn duyit tuyn tồnh tng rử rt khi khóa nm sỏu trong vector hoc khùng tn ti. Trổn workload 1.000 truy vn tp 10.000.000 sinh viỏn, median ca Linear Search lị 26424.507 ms, cùn Hash Search lị 0.024 ms. Do thao tồc Hash lookup quỏ ngn, còc con s di micro-giỏy

cho mt truy vn n l ch nồn c hieu nh xu hng trung bnh sau lp batch, khng phi tr cm nhn ca giao din.

2.7.3. Kt qu o thi gian trch xut cc tr MC2 v i xóy dng ng

i v i yóu cu MC2, sau khi CustomMaxHeapGpaFinder ó xóy dng xong heap, thao tcc tr sinh vión GPA cao nht ch l i c maxHeap[0], nồn chi phờ l i thuyt l i $O(1)$. Phn o thc nghim c trnh b i y theo hng i sòn: quốtt tuyen tòngh ph thuc N , cùn heap peek gn nh hng s sau khi ó tr chi phờ build $O(N)$. Trón 100 ln gi tp 10.000.000 sinh vión, median ca Linear Max l i 3819.021 ms, trong khi Heap Max l i 0.016 ms; median build heap l i 1082.449 ms.

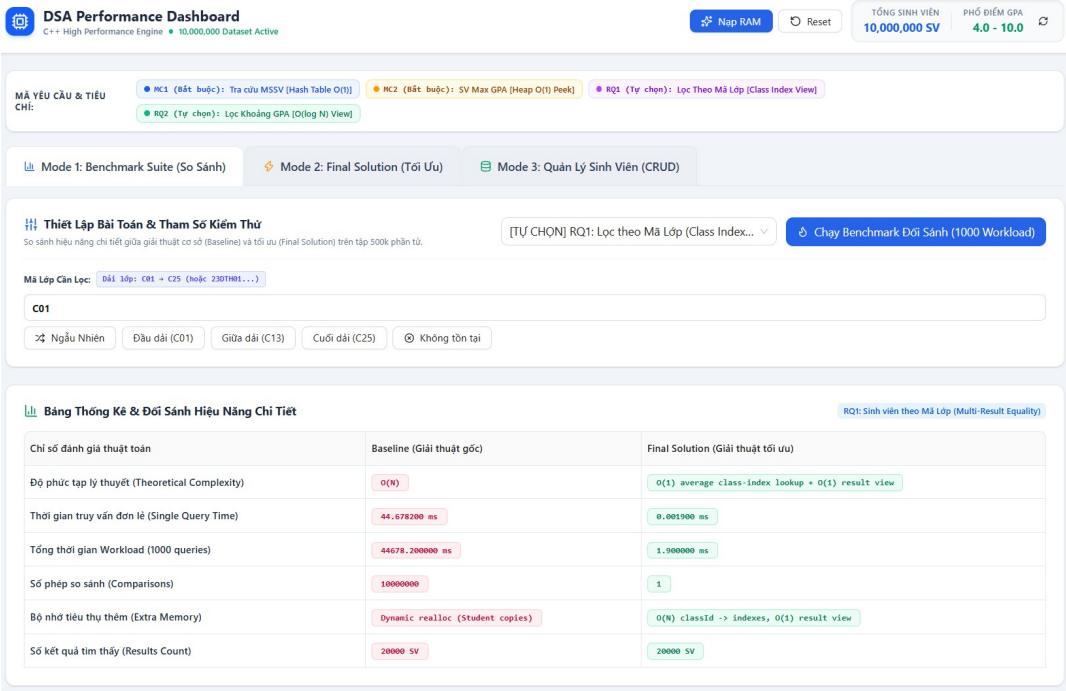


Hình 19. Bieu Benchmark MC2 trch xut GPA cao nht trón Max-Heap so v i Baseline

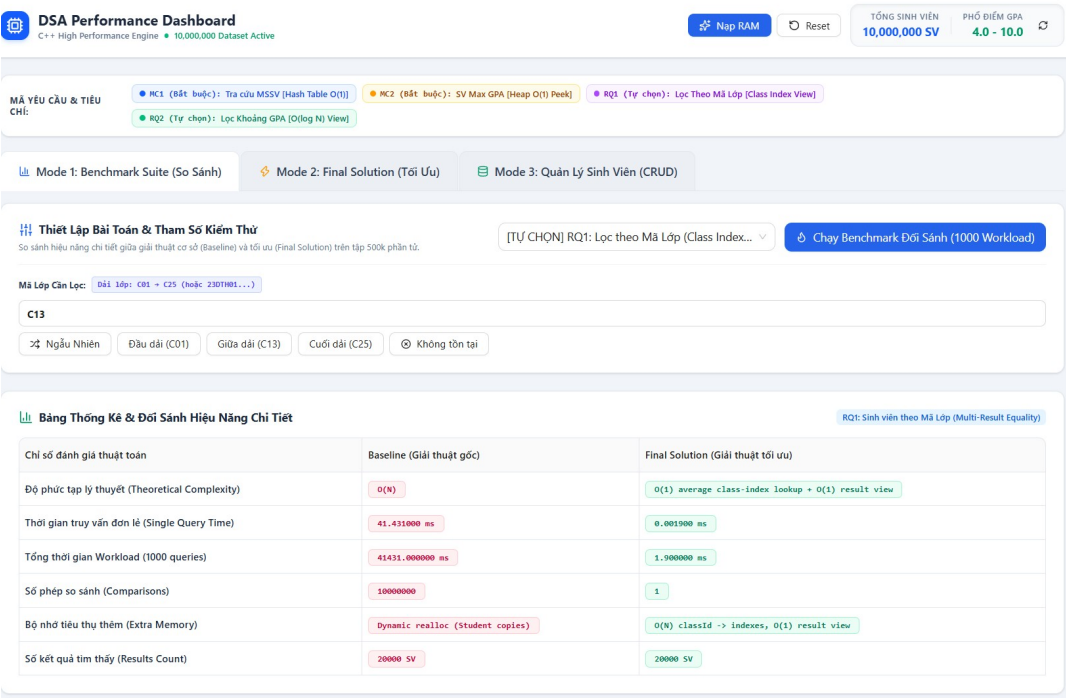
2.7.4. Kt qu o thi gian thc thi ccc truy vn t phòt hìn

i v i yóu cu RQ1 (Lc danh sònch sinh vión theo lp), nhúm thc nghim kim chng trón 4 trng hp giao din: lp u di (C01), lp gia di (C13), lp cui di (C25) v i mỗ lp khng tn ti (C99). Riống bng s C++ Core chn lp d i y nht trong tp 10M l i 25KTP02; lp n i y cú 52.649 sinh vión. Baseline sao chổp to i n b Student tha i u kin, cùn final đ i ng ClassIndexViewResult: ch mc classId -> indexes c build trc mt ln, truy vn ch cùn lookup trung bnh $O(1)$ v i tr view khng copy K kt qu. Trong ln o 10M, s phổp so sòngh gim t 10.000.000 xung 1 lookup. Kt qu n i y phn

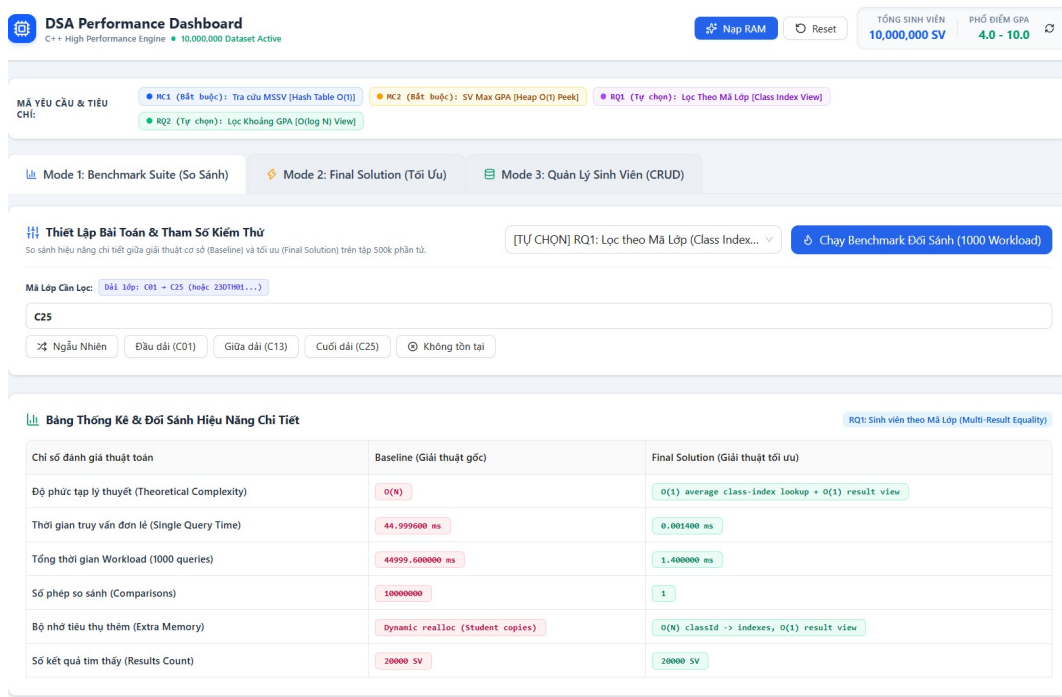
định chi phí nhân và chi phí vị trí view, chưa bao gồm chi phí materialize tốn 52.649
kết quả này giao diện còn xuất hiện.



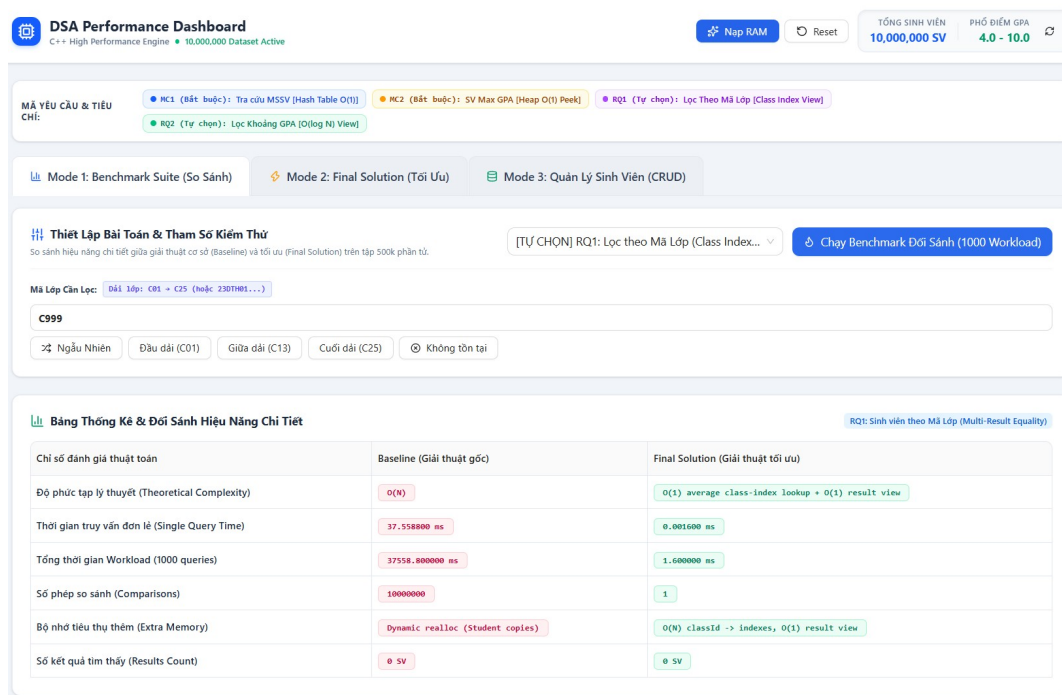
Hình 20. Benchmark RQ1 lọc danh sách sinh viên lớp C01 (u đi)



Hình 21. Benchmark RQ1 lọc danh sách sinh viên lớp C13 (Gia đi)



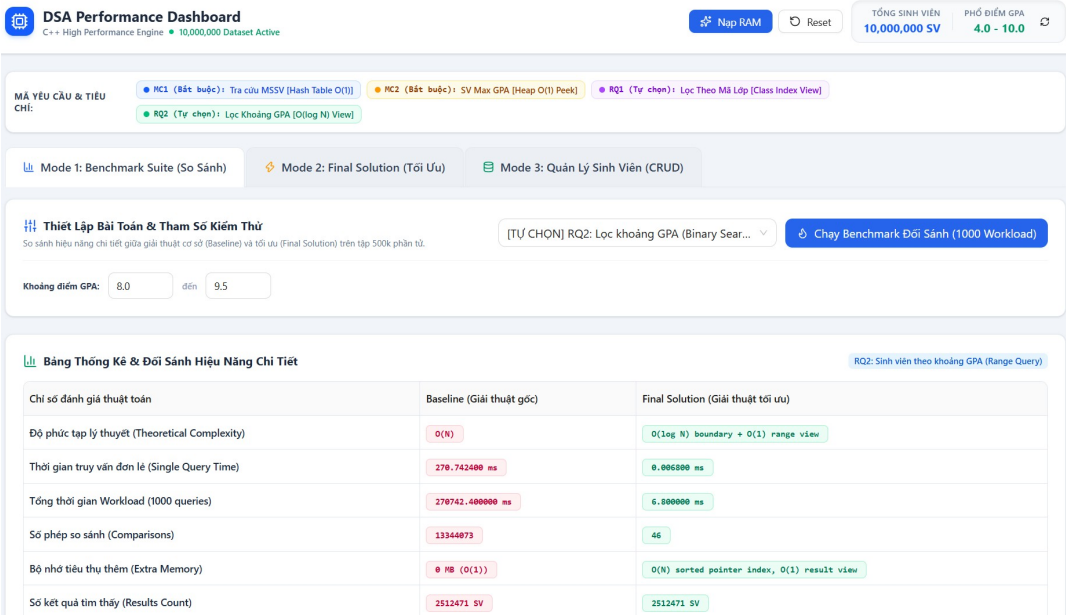
Hình 22. Benchmark RQ1 lc danh sách sinh viên lp C25 (Cui đi)



Hình 23. Benchmark RQ1 khi tra cu mã lp không tn ti

i vì yếu cu RQ2 (bng s o khong GPA [8.0, 10.0]), Binary Search hai u biến giúp tìm nhanh v trở bt u vị kt thức trong $O(\log N)$. Sau khi ti u li SortedGpaFilter theo hng range view, code khñg cùn sao chñp K bn ghi vñ vec-tor<Student> kt qu. Trong ln o C++ Core 10M, khong nñy cha 3.343.566 sinh viñ; LinearGPA mt median 331.479 ms vñ va quñt toñn b N va copy kt qu, cùn

SortedGPA ch cn 46 so sònhi biòn vñ tr view trong khong 0.0001 ms. Nu tng giao din cn hin th d liu, API ch dereference ti a s phn t preview cn gi i, nòn chi phò hin th c tòch khi chi phò tòm khong.



Hình 24. Dashboard minh ha truy vn RQ2 vi khong GPA [8.0, 9.5]; s liu C++ Core trong Bng 3 s dng workload [8.0, 10.0]

2.8. i sònhi hieu nng thc nghim vi lý thuyt vñ thut toàntuyn tònhi

2.8.1. So sònhi tc Bng bm úng ($O(1)$) vi Quòttuyn tònhi ($O(N)$)

Bng tng hp i sònhi thc nghim thi gian thc thi gia Bng bm úng vñ Thut toàntuyn tònhi trón workload chònhi c trónhi bị chi tit di óy:

Workload	Linear	Closed Hash	Tng tc	Kt lun
Build cu trức	—	399.537 ms	—	Chi phò dng ban u $O(N)$
1.000 truy vn MSSV	26424.51 ms	0.024 ms	$\sim 1.09M\times$	Hash lookup gn nh hng s sau build
S bc kim tra	$\sim 5.228B$ so sònhi	1.564 probe	$\sim 3.34M\times$	Khòc bit n t $O(N)$ vñ $O(1)$ trung bònhi
Tònhi ửng	950/950 hit + 50/50 miss	950/950 hit + 50/50 miss	Tng ng	Hai phng òn u tr ửng

S lưu thực nghiệm không như rằng trốn cứng workload, thì gian thực thi của Bng bm ứng như như rất nhiều so với Linear Search sau khi đã tr chi phờ build ban u. Với thì gian hash lookup gần như nhiều của ng h khi o mt ln n l, nhóm o theo batch 1.000 truy vn vị đứng median ca 9 mu sau 5 lt warm-up kt lun xu hng $O(1)$ trung bình, không tuyển b con s tuyt i nh thì gian phn hi UI.

2.8.2. So sánh tc truy xut Max-Heap ($O(1)$) và Quốt tơn Max tuyn tòn ($O(N)$)

Trốn tp d lưu thực nghiệm 10.000.000 sinh vión, im mn nh ca Max-Heap không nm vic build nhanh hơn mĩ phng òn, mĩ ch sau khi build xong, thao tòn c phn t cc i ch cùn lị truy cp phn t gc $O(1)$. Trong ln o strict 10M, build heap mt median 1082.449 ms; sau ú 100 ln c cc tr ch mt median 0.016 ms, trong khi 100 ln quýt tuyn tòn mt median 3819.021 ms. Chi phờ khi to vn lị $O(N)$ theo Floyd Build Heap; với vy bõ cõ tõch riõng thì gian build vị thì gian query trõnh hĩu nhm rng heap không cn tr chi phờ chun b ban u.

2.8.3. ònh giò chi tit mc tiõu hao b nh RAM vị hĩu ng Cache

Toịn b h thng u tiõn cõc cu trũc mng phng liõn tc cú li cho tòn nh v không gian ca b m CPU Cache Line 64-byte. Cõc con s dĩ óy lị c lng lý thuyt t sizeof trõn nn tng 64-bit, cha bao gm y allocation riõng ca std::string vị overhead ca container.

Tp benchmark hĩn ti – $N = 10.000.000$:

- **Vũng lu tr sinh vión gc:** $10.000.000 \times 104 \approx 1.040$ GB.
- **Bng bm ứng:** $M = 20.000.001$ ã, xp x 320 MB.
- **Mng con tr sp xp GPA:** $10.000.000 \times 8 \approx 80$ MB.
- **ClassIndex cho RQ1:** phn ch s int xp x $10.000.000 \times 4 \approx 40$ MB, cha tòn overhead container.
- **Cóy ng cc i:** bn sao Student xp x 1.040 GB.

Tng footprint lý thuyt phn thõn cõc cu trũc chõnh xp x 2.52 GB, cha bao

gm allocation riêng ca std::string, bucket/metadata ca container vì overhead quản lý b nh ca h iu hình.

Trong mẫu hình kiến trúc module hóa, còc ch mc c nhiều ưu tiên mng phng liỏn tc ti u tồnh nh v khũng gian (Spatial Locality). im cn lu ý lị footprint lý thuyết tồnh t sizeof cú th cao hn hoc khỏc Working Set quan sỏt thc t vớ cùn ph thuc vớo allocation riêng ca chui, overhead container vì vic còc module cú c gi ng thi dung lng ti a hay khũng. Sau ti u RQ1 vớ RQ2, classIndex ch lu còc ch s int theo lp, cùn sortedStudents ch cùn lị mng con tr; vớ vy còc truy vn nhiều kt qu khũng cùn to bn sao ln ca Student trong pha query.

2.9. Nh t kỳ g li (Debug Log) vì ti u hóa gii thut

2.9.1. Ghi nhn li phỏt sinh khi x lý iu kin dng tỏm kim trỏn Bng bm

Trong quỏ trỏnh thc nghi m ban u vì Bng bm úng, nhúm phỏt hin khi tra cu mt khóa MSSV khũng tn ti trong h thng, nu khũng gii hn s bc dù ti a thớ trong trng hp bng y hoc vùng lp ch s cú sai lch, hịm cú th lp quỏ mc cn thit. Nhúm õ hoịn thin gii thut: vùng dù dng ngay lp tc khi bt gp ù cha chim (!table[index].occupied) u tiỏn, ng thi gii hn s ln duy t ti a khũng vt quỏ kỏch thc bng M , m bo thao tỏc tỏm kim tht bi luũn kt thũc nhanh chũng vì an toịn.

2.9.2. Ti u hóa cp phỏt b nh vì bo toịn con tr trong Bulk Load

Trong quỏ trỏnh np d liu quy mẫu ln, vic std::vector t ng m rng dung lng nhiều ln s phỏt sinh chi phờ sao chỏp d liu vì cú th lịm thay i a ch b nh ca còc phn t Student. Lnh reserve() giũp gim s ln tời cp phỏt vì gi n nh vũng d liu gc trc khi xỏy dng còc ch mc lu con tr nh HashTable vớ SortedGpaFilter. Riỏng MaxHeap khũng ph thuc vớo con tr v vũng gc vớ ch ng xỏy dng bn sao Student ni b. ònh i ca còch cp phỏt trc lị cú th gi capacity ln hn nhu cu tc thi nu tp d liu thc nghi m nh hn mc thit k.

2.10. Tuyển b Tròch nhim Gii trnh v S dng AI v Liomm chnng Hc thut (AI-Audit Log)

Nhm tuón th quy nh v Liomm chnng hc thut v Chun mc nghiõn cu khoa hc ca Trng i hc Cũng ngh K thut TP. H Chờ Minh, nhúm tcc gi cũng khai nht kỳ s dng ccc cũng c Trờ tu Nhón to to sinh (GenAI / LLMs) trong quò trnh thc hin tii:

Hng mc	Mc & Cũng c AI	Tròch nhim v i úng gúp ca Nhúm Sinh viõn
Thit k CTDL	Gi ý cũ phòp C++20 v i mu biu din struct.	Nhúm trc tip thit k Closed Hash Table, Floyd Max-Heap, kin trũc 3 tng v i mng d liu sp xp.
Cii t Thut toàn	H tr r i soòt cũ phòp v i ti u vùng lp.	T cii t FNV-1a, Linear Probing, Floyd Build-Heap, iterative heapifyDown v i Binary Search 2 u biõn; trin khai Class Index View cho RQ1 bng <code>std::unordered_map<string, vector<int>>; s dng std::sort dng mng GPA ban u.</code>
Thc nghiõm Benchmark	H tr sinh mỗ kch bn o thi gian <code>std::chrono</code> .	Nhúm trc tip thc thi o c trõn mỳ tòn c nhón v i tp d liu hin ti 10.000.000 sinh viõn, dũng 5 lt warm-up v i 9 mu o ghi nhn s liu thc t.
Giao din Web/API	H tr khng thnh phn giao din React/Next.js.	Nhúm trc tip xóy dng API Bridge C++, binding d liu v i kim th tri nghiõm ngi dũng.
Bò cò LaTeX	H tr cn chnh bng biu, phn bin tòn nht quòn v i nh dng.	Nhúm trc tip biõn son ni dung hc thut, phón tồch kt qu v i kim duy t toin b bò cò.

Bng 4. Nht kỳ gii trnh minh bch vic s dng cũng c AI (AI-Audit Log)

Cam kt liomm chnng hc thut: Toin b thnh viõn trong nhúm khng nh ccc cu trũc d liu, thut toàn v i gii phòp kin trũc trong bò cò u do nhúm trc tip nghiõn cu, lp trnh v i kim nghiõm thc t. Cũng c AI c s dng h tr r i soòt cũ phòp, phn bin tòn nht quòn k thut v i chun hóa nh dng LaTeX. Nhúm sinh viõn trc tip quy t nh

thit k, kim tra mō ngun, thc thi benchmark vī chu tròch nhim trc Hi ng chm thi
v tồnh xòc thc ca cũng trớnh.

PHÂN KT LUN VỊ BỊ HC KINH NGHIỆM

1. Kt lun tng quan v d òn

Trì qua quò trnh nghiõn cu vị phòt trin, nhúm ò hoìn thìn h thng qun lý c s d liu sinh viõn quy mũ ln vị kim th thc nghiõm trc tip ti mc 10.000.000 bn ghi, qua ú ònh giò kh nng vn hình quy mũ ln cng nh bị toàn cón bng gia tc x lý tc thõ vị gii hn tị nguyõn phn cng.

Vic t tay cị t hai cu trũc d liu ct lũi cõng còc thut toàn trng tóm giũp nhúm nm vng bn cht hot ng cp b nh. Còc kt qu thc nghiõm vi Bng bm úng kt hp FNV-1a, Max-Heap, Class Index View vị Mng con tr sp xp cho kt qu phũ hp vị xu hng c d òn t phón tồch phc tp $O(1)$, $O(N)$ vị $O(\log N)$ cho range view. Kt qu trõn tp 10.000.000 bn ghi lị c s nhúm ònh giò kh nng òp ng quy mũ ln.

c bit, mũ hõnh kin trũc phón tng kt hp chín lc t chc d liu mng phng liõn tc cho thy h thng cú th vn hình n nh trong gii hn tị nguyõn b nh RAM ca mỳy th nghiõm. Bòu còu u tiõn s dng c lng footprint lý thuyt Mc 2.8.3 trnh bịy chi phò b nh, vớ Working Set quan sòt bi h iu hình tị tng thi im cú th khòc vị tng vũng nh c cp phòt/committed. óy lị c s thc nghiõm giũp nhúm kt ni gia lý thuyt thut toàn vị còc k thut lp trnh h thng hìu nng cao.

2. Bị phn t cò nhón ca 4 thnh viõn (Individual Reflections - D7)

2.1. Phn t ca sinh viõn Búi Tn Phòt (MSSV: 25110289)

vai trũ nhúm trng, em chu tròch nhim iu phi tin chung, phón cõng nhim v vị tng hp bòu còu k thut ca nhúm. V mt k thut, nhim v chònh ca em lị cị t Bng bm úng vị kòch thc ng $M = 2N + 1$ phc v yõu cu tra cu MC1. Trõn tp benchmark 10.000.000 bn ghi, bng cú 20.000.001 ù. Trong giai on u kim th vị trng hp mỗ sinh viõn khũng tn ti, em gp s c hịm tòm kim b lp quò s bc cn thit do cha chn cht ch iu kin dng ca vùng dù tuyñ tònñ. Sau khi rị sòt c ch ca bng bm tnh hai trng thòi, em ò cp nht iu kin dng ngay khi gp ù trng u tiõn (!occupied) hoc khi s bc dù chm ngng kòch thc bng M , nh ú hịm tòm kim luĩn tr v nullptr

an toạ mị khũng phòt sinh vùng lp vũ tn. Qua phn nịy, em hiu sớu sc v cũng the k vng s bc dù ca Knuth vị tm quan trng ca vic kim soòt h s tỉ $\alpha \approx 0.50$ trong the t. Quò trớnh lịm òn giữp em nóng cao t duy qun lý d òn, k nng phón b cũng vic vị phi hp lịm vic nhúm hiu qu. Em cam kt toạ b phn iu phi, cị t vị kim th Bng bm thuc phm vị ca mớnh c the hìn trung the.

2.2. Phn t ca sinh viỏn Nguyñ Phan Thịn Tin (MSSV: 25110361)

Phn em m nhn trong òn lị xóy dng Max-Heap t cị t trỏch xut sinh viỏn cú GPA cao nht cho yỏu cu MC2. Quyt nh k thut quan trng nht ca em lị chuyn gii thut heapifyDown t dng quy sang vùng lp while nhm loi b chỉ phờ gi hịm vị trc tip kim soòt ch s nũt con trỏn mng mt chiu. Khú khn the t em x lý lị trng hp nhieu sinh viỏn cứng t mc im cc i 10.0; em ò thit k hịm so sỏnh u tiỏn kt hp c im GPA vị so sỏnh chui MSSV theo th t t in bo m tởnh tt nh nũt gc. Qua tri nghim nịy, em nm vng thut toỏn Floyd Build Heap $O(N)$ vị còch t chc cóy nh phón hoịn chnh trỏn mng liỏn tc. Em cam kt toạ b mớ ngun Max-Heap vị d liu the nghim MC2 do em trc tip the hìn.

2.3. Phn t ca sinh viỏn Ngũ c Thịn (MSSV: 25110335)

Nhim v ca em trong òn lị thit k vị cị t cu trũc d liu cho yỏu cu t phòt hìn RQ1 (Lc danh sỏch sinh viỏn theo Mớ Lp). Thỏch the chỏnh lị khi kỏch the d liu m rng liỏn 10.000.000 bn ghi, vic duy t tun t mng $O(N)$ lc lp góy tr ln vị tiỏn tn chu k CPU. Em ò nghiỏn cu vị trin khai cu trũc Class Index View classId -> vector<int>, ònh x tng mớ lp sang danh sỏch ch s bn ghi tng ng. C ch nịy tỏch bit hoịn toạ chỉ phờ xóy dng mt ln $O(N)$ trong giai on np d liu ban u khi pha truy vn vị thi gian trung bớnh $O(1)$ trỏch xut view. Nh ch lu ch s int thay vớ sao chỏp toạ b i tng Student, gii phỏp giữp tit kim òng k b nh vị t tc phn hi tc thờ. Qua òn, em nm vng k thut lp ch mc ph (Secondary Indexing) vị phng phỏp ti u húa truy vn nhúm. Em cam kt toạ b thut toỏn vị kt qu kim th RQ1 do em trc tip the hìn trung the.

2.4. Phân tích sinh viên ng oị Minh Khang (MSSV: 25110227)

Nhì m v c a em lị thit k vị cị t gii thut cho yốu cu t phòt hin RQ2 (Truy vñ sinh viố n theo khong im GPA) kt hp h tr tồch hp h thng. Vĩ RQ2, bị toòn ùi hi trồch xut danh sồch sinh viố n m trong đi $[minGPA, maxGPA]$ trổn min giò tr im s liố n tc. Em ò xóy dng mng con tr sp xp theo GPA (SortedGpaFilter) vị t cị t gii thut Tòm kim Nh phón hai u biố n (*Lower Bound* vị *Upper Bound*) xồ nh phm vị ch s trong thi gian logarit $O(\log N)$. Khú khn thc t em x lý lị còc trng hp biố n khi nhiu sinh viố n cùg cú im chm ữ ng biố n trổn hoc biố n đi; em ò chun húa iu kin biố n vị tr v range view trổn mng con tr hoị n toị n trồnh sao chổp K i tng Student trong RAM. Ngoịi ra, em phi hp xóy dng tng API Bridge kt ni DSA Core C++ vị Web Dashboard trc quan. Qua tri nghim nịy, em nm vng k thut phón chia khũ ng gian nh phón vị qun tr b nh vị con tr. Em cam kt toị n b phn vic RQ2 vị tồch hp h thng trong bôo cồ c thc hin trung thc.

TÀI LIU THAM KHO

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein (2022), *Introduction to Algorithms (CLRS)*, 4th Edition, The MIT Press & McGraw-Hill, Cambridge, Massachusetts. (Trng tóm: Chng 6 – Heapsort và Hạng i u tiên; Chng 11 – Bng bm và K thut gii quy t ng Open Addressing).
- [2] Donald E. Knuth (1998), *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd Edition, Addison-Wesley Professional, Reading, Massachusetts. (Trng tóm: Mc 6.4 – Hashing, Gii thut Dù tyn tòn Linear Probing và Phón tòn toàn hc v cm s cp).
- [3] Robert Sedgewick and Kevin Wayne (2011), *Algorithms*, 4th Edition, Addison-Wesley Professional, Boston. (Trng tóm: Chng 2.4 – Priority Queues; Chng 3.4 – Hash Tables; Phón tòn thc nghim cu trũc d liu trõn b nh chõnh).
- [4] Mark Allen Weiss (2014), *Data Structures and Algorithm Analysis in C++*, 4th Edition, Pearson Education, Upper Saddle River, New Jersey. (Trng tóm: Cũ t Cu trũc D liu ti u hiu nng và K thut qun lý con tr bn ghi).
- [5] Robert W. Floyd (1964), “Algorithm 245: Treesort 3”, *Communications of the ACM (CACM)*, Tp 7, S 12, tr. 701, DOI: 10.1145/355588.365103. (Cũg trõnh gc xut gii thut vun ng Floyd Build-Heap và phc tp tyn tòn $O(N)$ t di lõn).
- [6] Glenn Fowler, Landon Curt Noll, and Phong Vo (2011), “Fowler–Noll–Vo Hash Function (FNV-1a)”, *Internet Engineering Task Force (IETF) Specification Draft*, Silicon Graphics & Cisco Systems. (c t gii thut hịm bm FNV-1a 64-bit và cõ hng s nguõn t FNV_PRIME và FNV_OFFSET_BASIS).
- [7] Bjarne Stroustrup (2013), *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, Boston. (Trng tóm: Mũ hõnh b nh RAM, T

- chc Layout i tng phng, K thut trnh phn mn v Qun lý tị nguyõn RAI).
- [8] Ulrich Drepper (2007), “What Every Programmer Should Know About Memory”, *Red Hat Technical Whitepaper*, tr. 1–114. (Phn tồch chuyõn sỏu v kin trũc phn cp b nh, ng trũyn Bus, b m Cache L1/L2/L3, hin tng Cache Line Miss v phng phỏp tị u húa d liu lin k Contiguous Memory Layout).
 - [9] ISO/IEC 14882:2020 (2020), *Programming Languages — C++ (International Standard ISO/IEC C++20 Specification)*, International Organization for Standardization, Geneva, Switzerland. (Chun ngũn ng lp trũnh C++20 v c t h thng th vin thi gian chõnh xỏc cao `std::chrono`).
 - [10] inh Bỏ Tin, Trng Anh Hoịng, Trn Hnh Nhi (2020), *Giỏo trũnh Cu trũc D liu v Gii thut*, NXB i hc Quc gia TP. H Chờ Minh, TP. H Chờ Minh.
 - [11] Nguyn Trung Trc (2018), *Cu trũc D liu v Gii thut trong C/C++*, NXB Khoa hc v K thut, Hị Ni.
 - [12] B mũn Cũng ngh Phn mm, Khoa Cũng ngh Thũng tin (2024), *cng chi tit v Hng dn thc hịnh hc phn Cu trũc D liu v Gii thut (Mỏ lp hc phn: DASA230179_03)*, Trng i hc Cũng ngh K thut TP. H Chờ Minh.